

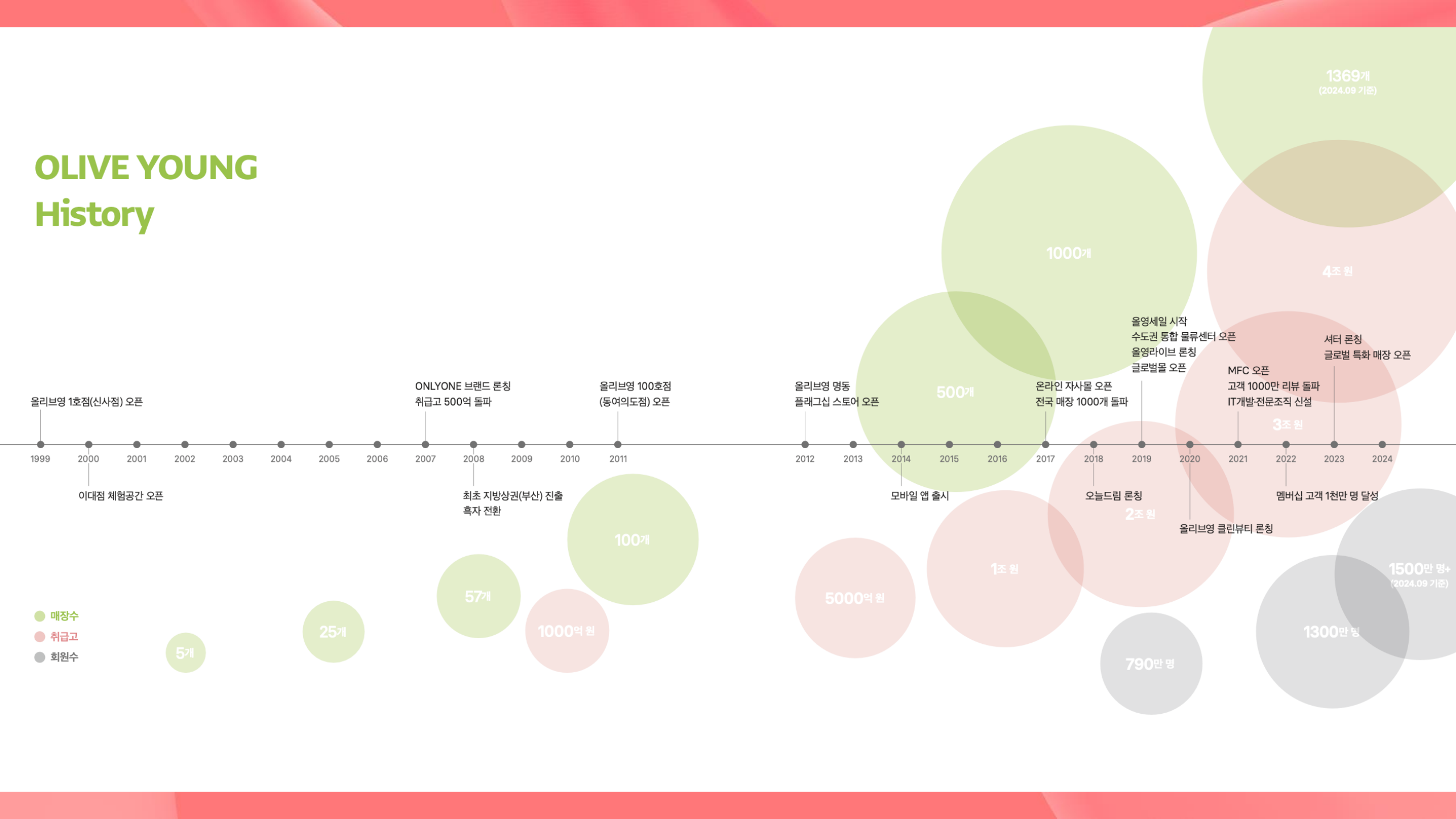
올리브영 물류시스템 개선기: 옴니채널 실시간성과 확장성을 잡다

IMPROVING LOGISTICS SYSTEMS AT OLIVEYOUNG:
ENABLING REAL-TIME AND SCALABLE OMNICHANNEL FULFILLMENT

김진배님 | 이정표님

올리브영 테크플랫폼센터 백엔드 엔지니어

OLIVE YOUNG History

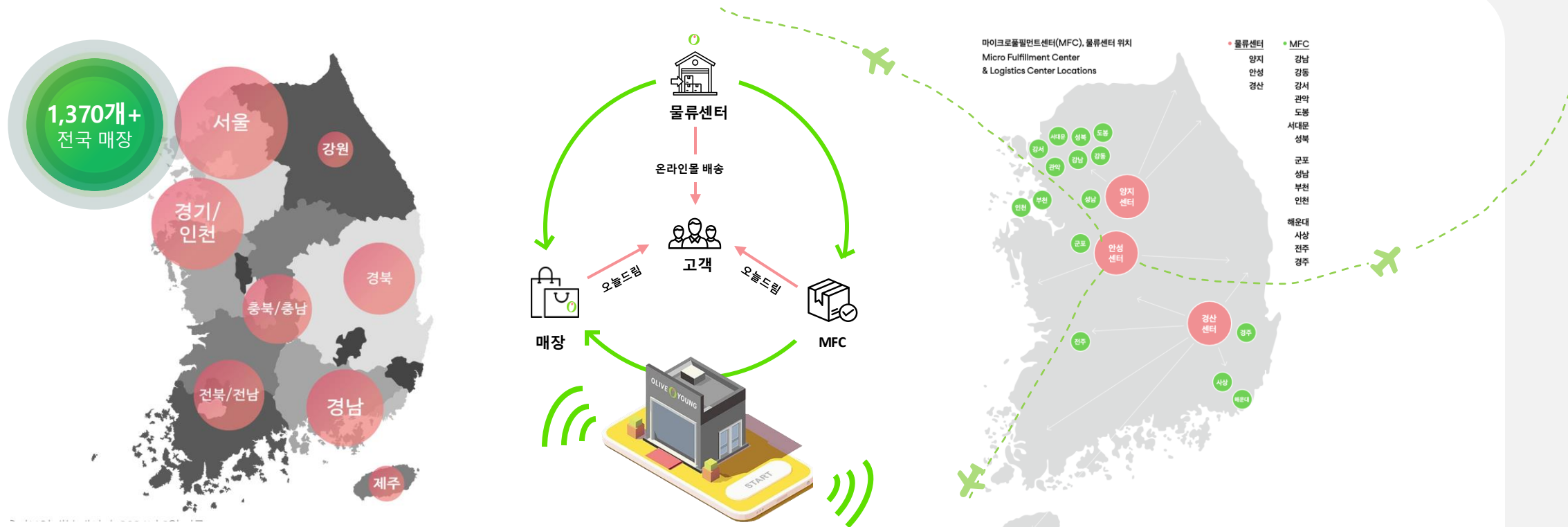


국내 대표 뷰티&헬스 트렌드 리딩 플랫폼, 올리브영



매장/브랜드/글로벌	+	온라인	+	옴니채널
1,370개+ 매장 국내 최대 직영 매장 네트워크		820만+ 월간 활성 사용자 모바일 앱 MAU		5조원+ 올리브영 취급고 총 상품 거래 규모 확장
2,400개+ 브랜드사 함께 상생하는 협력사 증대		99.94% 서비스 가동률 시스템 가용성 확대		1.6천만+ 멤버십 높은 충성도의 코어 이용자 확보 (국내 인구의 약 20%)
6.4만개+ 운영 상품 2.4천개+ 브랜드의 상품 수		16만+ 최대 동접 동시 사용자 최대 수용		2.3억건+ 오늘드림 주문 빠른 배송 누적 주문 건수
10개 자체 브랜드 올리브영 PB 브랜드 수		3,400개+ 누적 상품 리뷰 온라인 리뷰 합산 건수		55분 오늘드림 리드타임 주문부터 배송까지 평균 시간
151% 구매 성장률 (YoY) 국내 방문 외국인 고객 구매건		3,140만+ 라이브 누적 시청 라이브 커머스 시청자 수		62% 3시간 이내 배송 커버리지 '오늘드림' 배송 커버리지

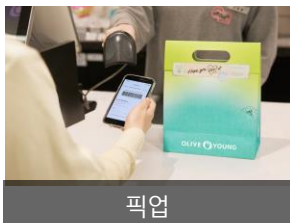
올리브영의 압도적인 옴니채널 인프라



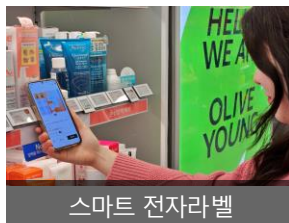
- * 물류센터: 재고 통합과 주문 이행의 중심 인프라로, MFC 및 매장과 연계된 분산 배송 체계를 통해 옴니채널 운영의 안정성과 확장성을 뒷받침함
- * 도심형물류센터(MFC): 도심이나 소비자에게 가까운 곳에 위치하여, 주문 처리 시간을 단축시키고 배송까지의 거리를 줄여줌으로써 쿠팡마켓플레이스 시장에서 경쟁력 확보 가능
- * 온라인물/모바일앱: 상품 탐색, 구매, 리뷰 등 고객의 전체 쇼핑 여정을 디지털로 구현하여, 매장 기반 서비스와 실시간 연동을 통해 일관된 경험을 제공

올리브영의 옴니채널 전략 및 서비스

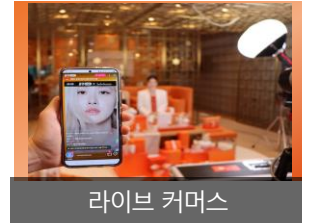
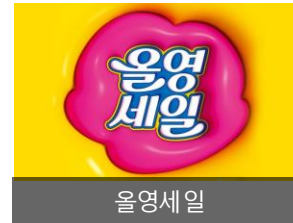
Core Experience 핵심 고객과의 접점



Omni Integration 옴니채널 전략 실행



Engagement & Content 브랜드 경험 및 콘텐츠 유도





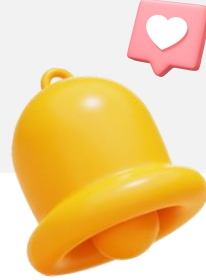
안녕하세요, 저는 김올영이라고
합니다.

저는 오늘 김진배님과 이정표님을
도와, 여러분이 **물류 도메인**을 더
쉽게 이해할 수 있도록 전문용어 및
주요 상황에 대한 설명을 보조해
드릴게요! 🙌 🙌 🙌



그리고 오늘 저희 세션을 집중해서
들으신 후 올리브영 부스를 방문해
질문 및 응원을 남겨주세요.

연사와의 시간에 이어 16시에는
별도의 추첨을 통해 소정의 선물도
드릴 예정이니 많은 방문 부탁드립니다!



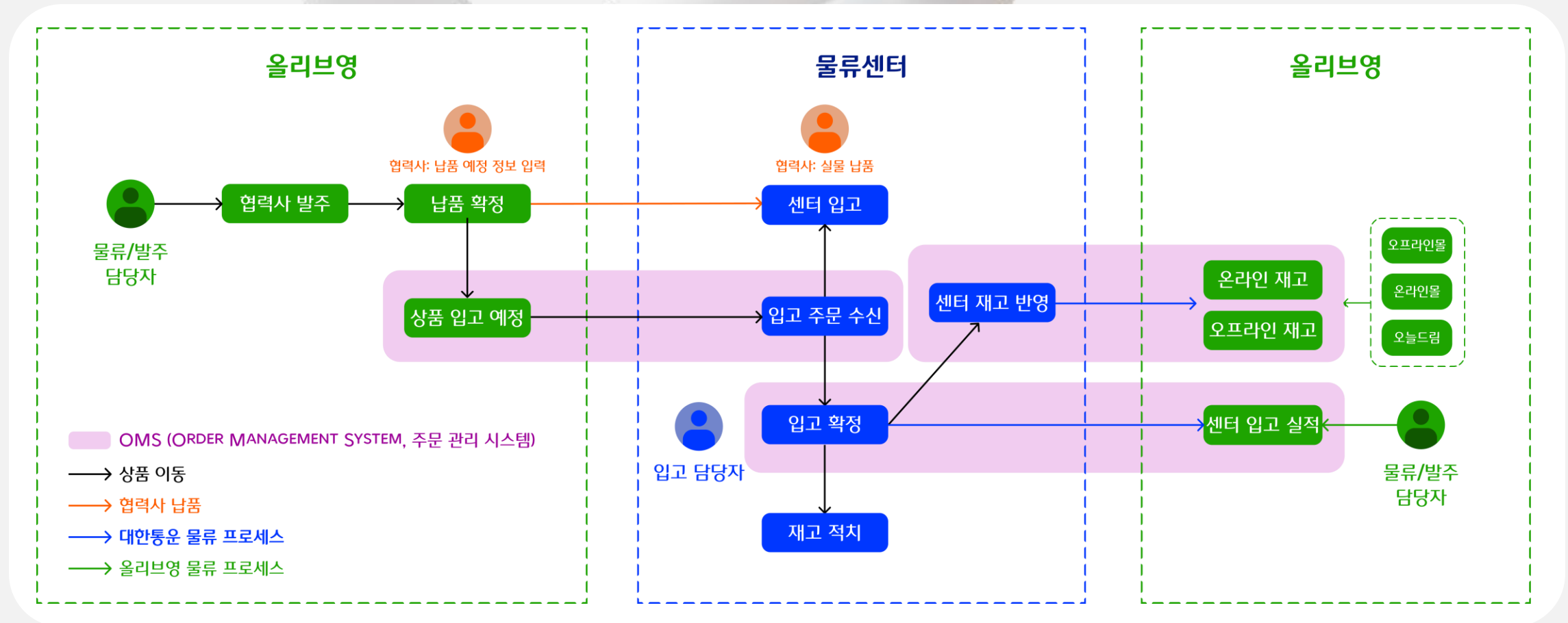
CHAPTER.1

실시간성과 확장성을 위한 데이터 연동

DATA INTEGRATION FOR REAL-TIME AND SCALABLE ARCHITECTURE

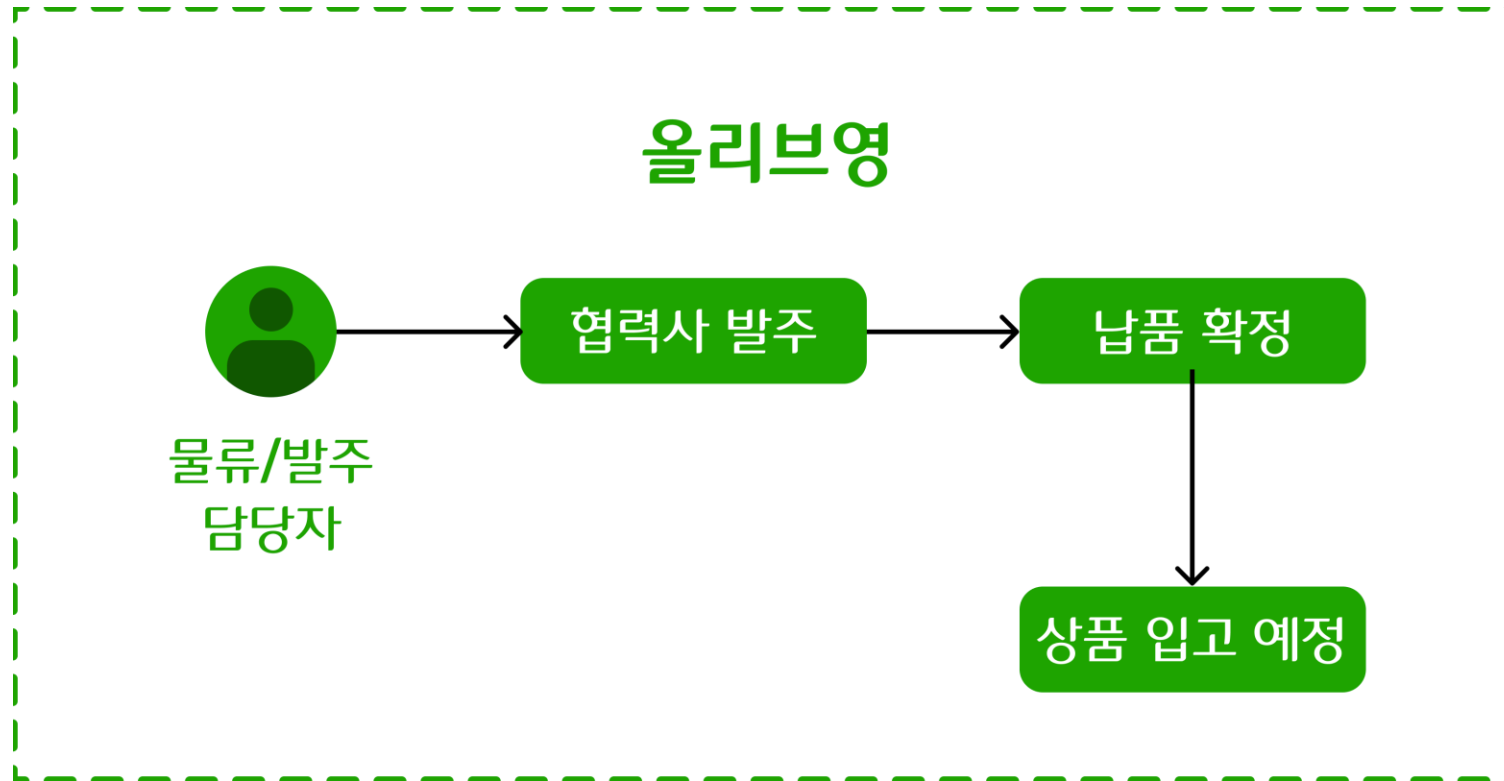
1-1. 발주부터 재고가 반영되기까지의 물류 프로세스

원활한 상품의 수급과 정확한 재고 관리를 위해 발주-입고-재고 반영까지 주문 관리 시스템(OMS) 동작



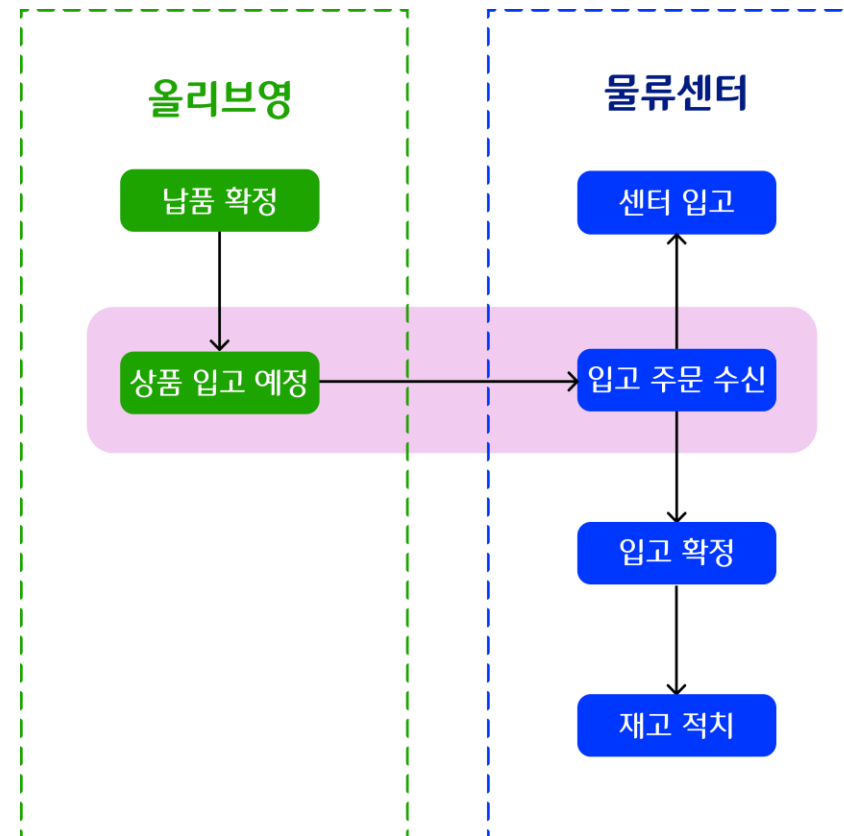
1-1. 발주부터 재고가 반영되기까지의 물류 프로세스

원활한 상품의 수급과 정확한 재고 관리를 위해 발주-입고-재고 반영까지 주문 관리 시스템(OMS) 동작



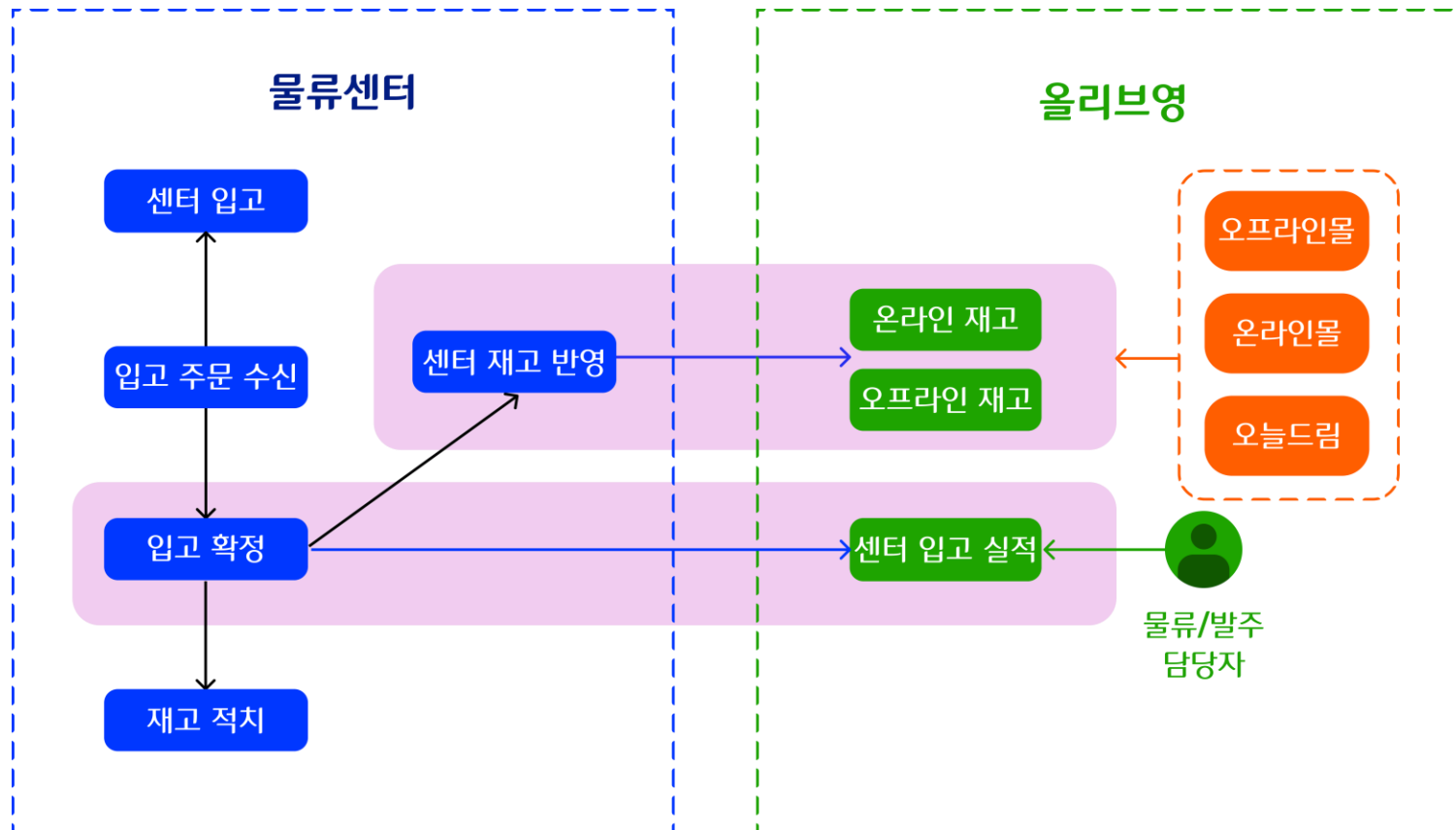
1-1. 발주부터 재고가 반영되기까지의 물류 프로세스

원활한 상품의 수급과 정확한 재고 관리를 위해 발주-입고-재고 반영까지 주문 관리 시스템(OMS) 동작



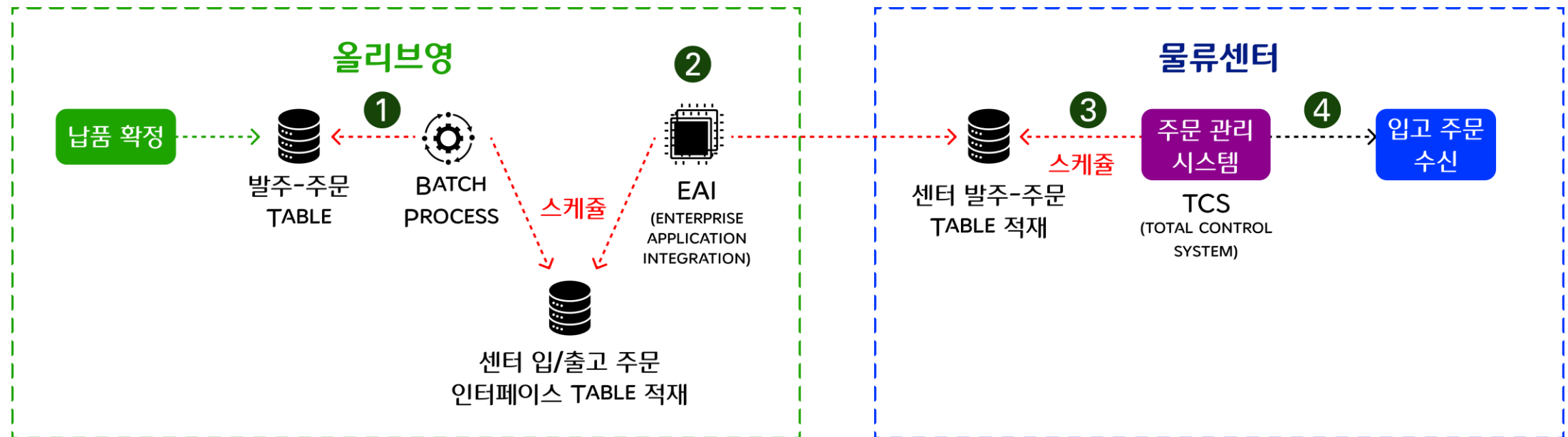
1-1. 발주부터 재고가 반영되기까지의 물류 프로세스

원활한 상품의 수급과 정확한 재고 관리를 위해 발주-입고-재고 반영까지 주문 관리 시스템(OMS) 동작



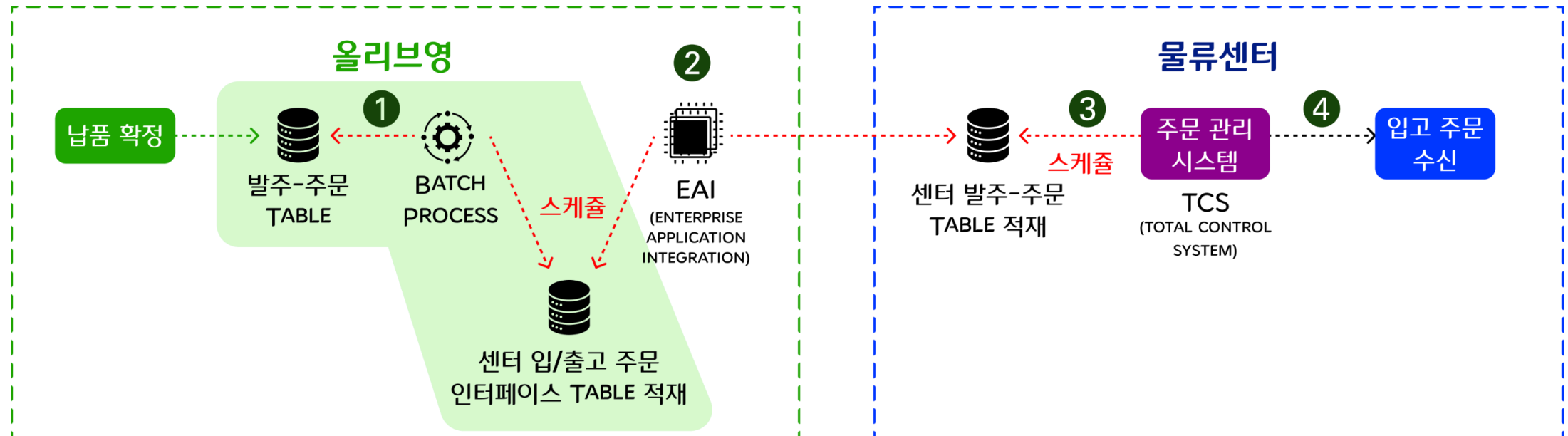
1-2. 발주-입고 데이터 연동 개선 전: 데이터 지연

배치(BATCH)와 엔터프라이즈 애플리케이션 통합(EAI)이 각각 30분 주기로 실행되어, 데이터가 올리브영 물류센터로 연동되기까지 최소 1시간의 지연 발생



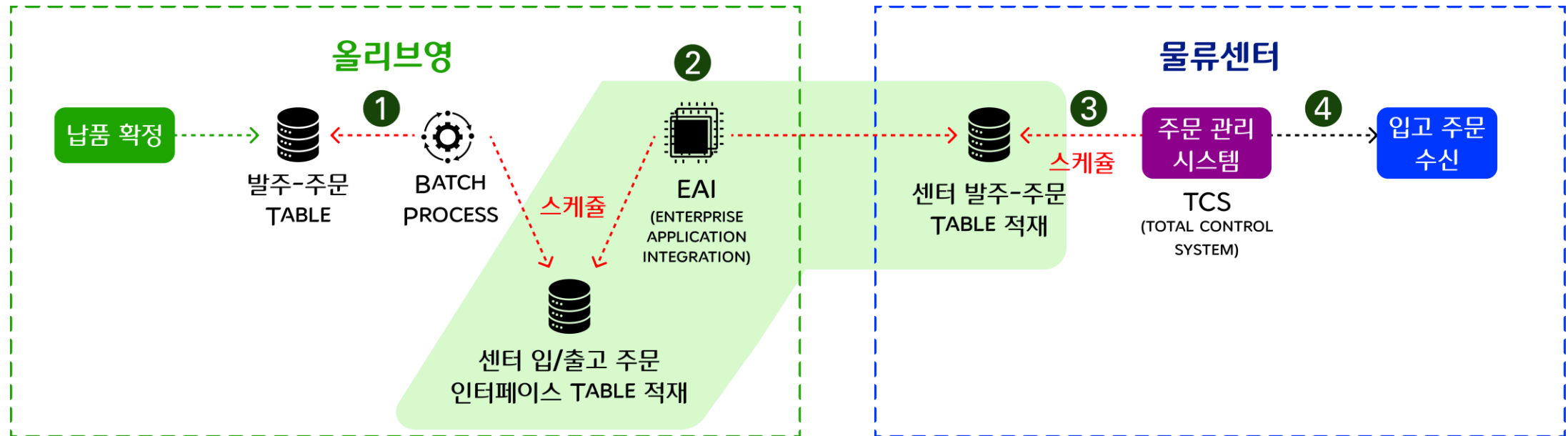
1-2. 발주-입고 데이터 연동 개선 전: 데이터 지연

배치(BATCH)와 엔터프라이즈 애플리케이션 통합(EAI)이 각각 30분 주기로 실행되어, 데이터가 올리브영 물류센터로 연동되기까지 최소 1시간의 지연 발생



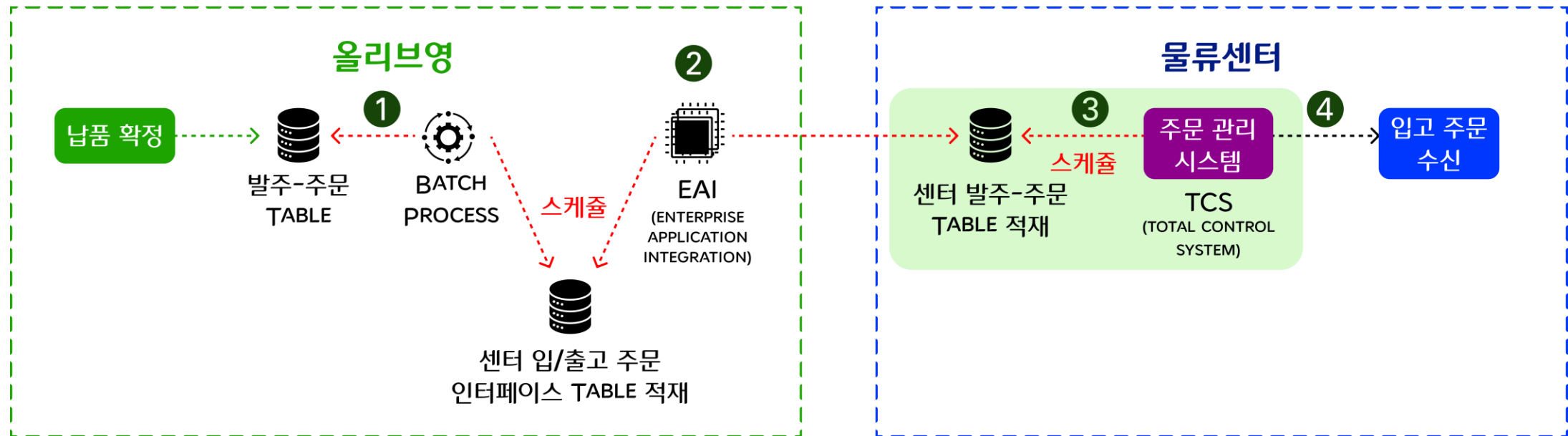
1-2. 발주-입고 데이터 연동 개선 전: 데이터 지연

배치(BATCH)와 엔터프라이즈 애플리케이션 통합(EAI)이 각각 30분 주기로 실행되어, 데이터가 올리브영 물류센터로 연동되기까지 최소 1시간의 지연 발생



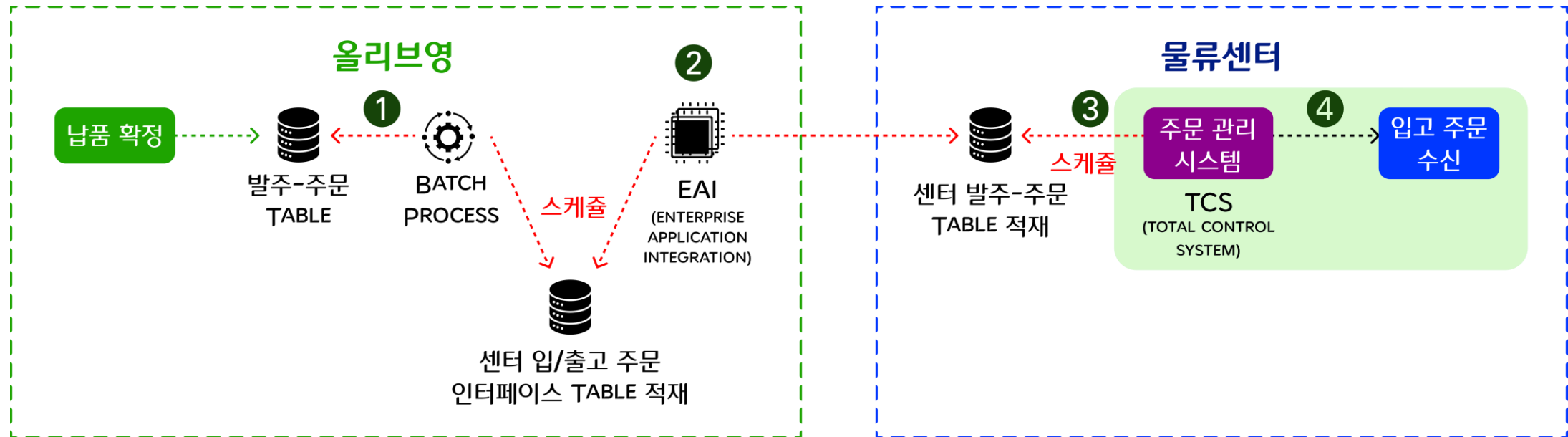
1-2. 발주-입고 데이터 연동 개선 전: 데이터 지연

배치(BATCH)와 엔터프라이즈 애플리케이션 통합(EAI)이 각각 30분 주기로 실행되어, 데이터가 올리브영 물류센터로 연동되기까지 최소 1시간의 지연 발생



1-2. 발주-입고 데이터 연동 개선 전: 데이터 지연

배치(BATCH)와 엔터프라이즈 애플리케이션 통합(EAI)이 각각 30분 주기로 실행되어, 데이터가 올리브영 물류센터로 연동되기까지 최소 1시간의 지연 발생



1-2. 발주-입고 데이터 연동 개선 전: 데이터 지연

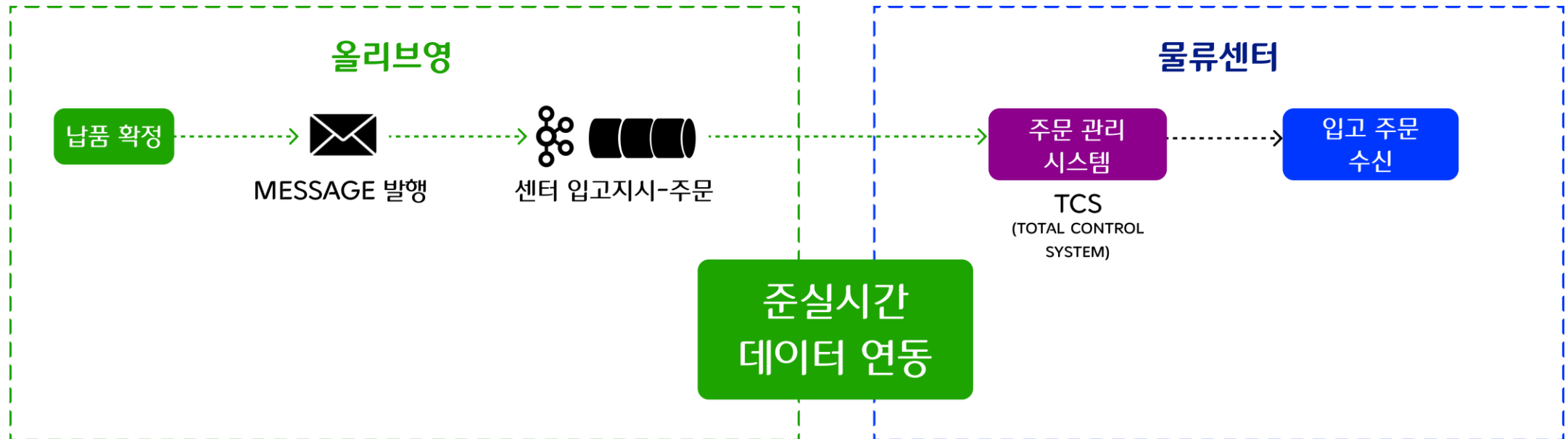
배치(BATCH)와 엔터프라이즈 애플리케이션 통합(EAI)이 각각 30분 주기로 실행되어, 데이터가 올리브영 물류센터로 연동되기까지 최소 1시간의 지연 발생



'결품(Out of Stock)'이란 '품절(Sold Out)'과 다른 의미이며, 재고부족으로 인하여 배송 또는 재고가 없는 경우를 말합니다.

1-3. 발주-입고 데이터 연동 개선 효과

KAFKA 도입으로 BATCH/EAI 기반의 지연 구조를 제거하고, 준실시간 데이터 흐름을 구현하여 데이터 연동 속도와 데이터 신뢰도 대폭 향상



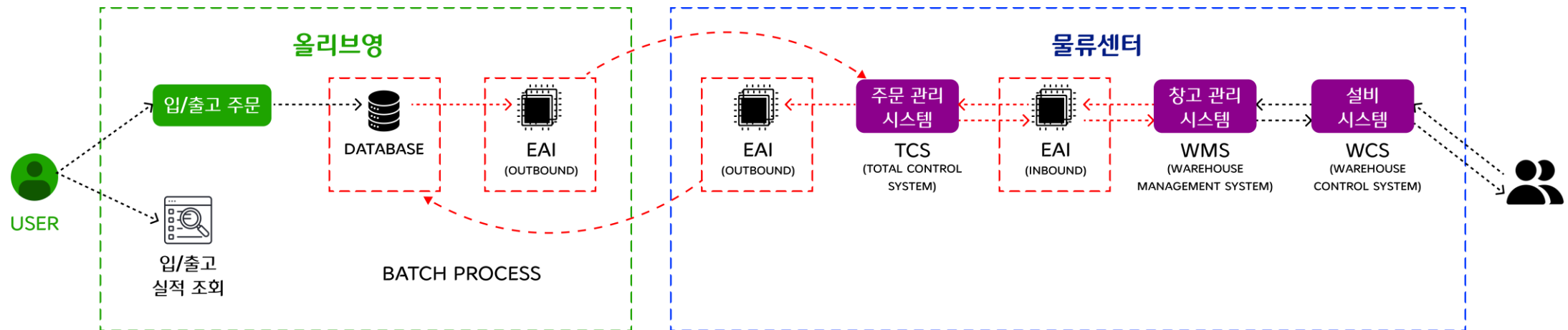
1-3. 발주-입고 데이터 연동 개선 효과

KAFKA 도입으로 BATCH/EAI 기반의 지연 구조를 제거하고, 준실시간 데이터 흐름을 구현하여 데이터 연동 속도와 데이터 신뢰도 대폭 향상



2-1. 입고-출고 주문 연동 개선 전

BATCH & EAI 기반의 순차적 연동 구조로 인한 한계점 존재

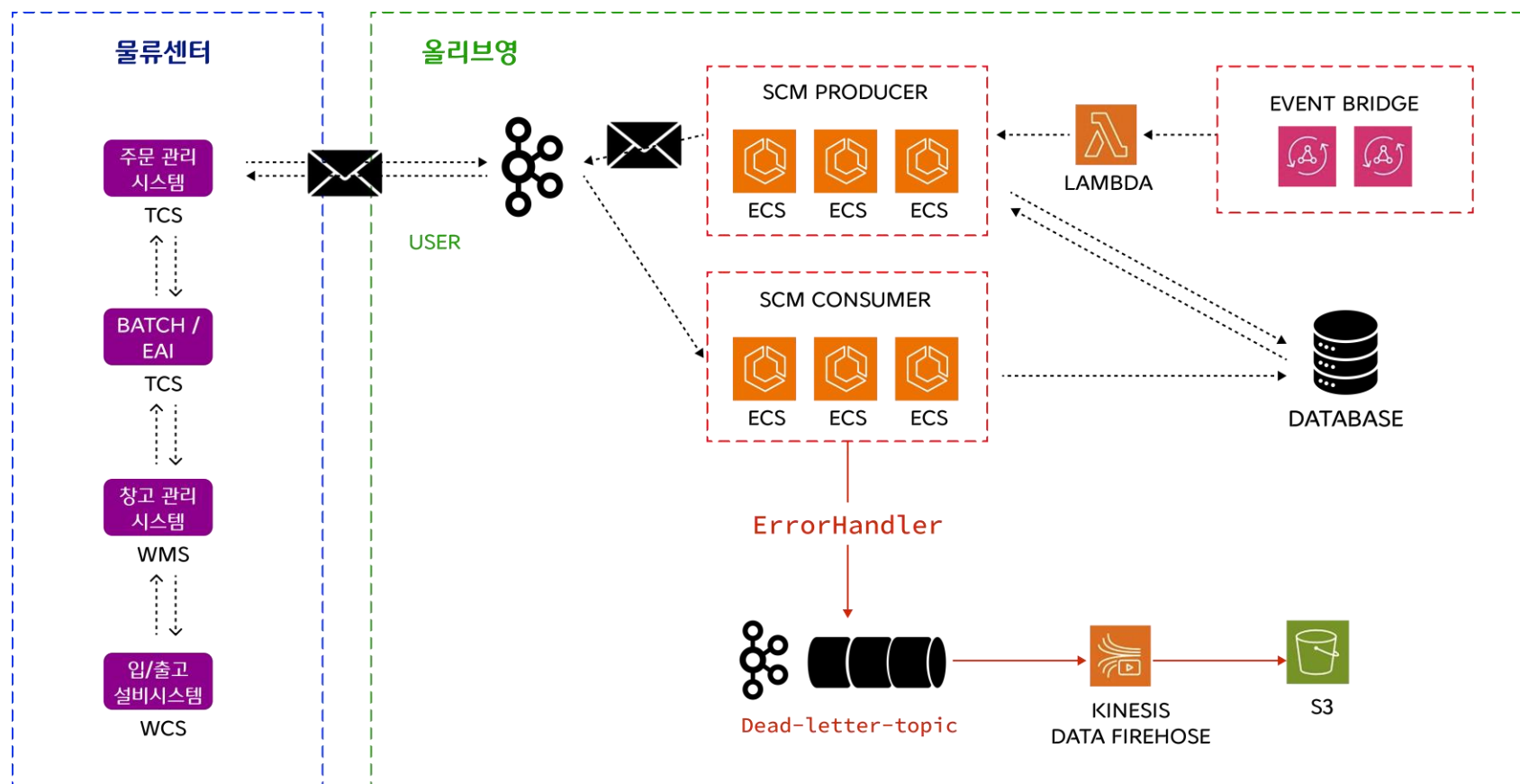


BATCH & EAI 연동 방식의 주요 한계

- 실시간성 미흡으로 정합성 보장 어려움
- 성능 저하 및 시스템 부하 유발
- 장애 복구의 비효율성
- 운영 복잡성 증가

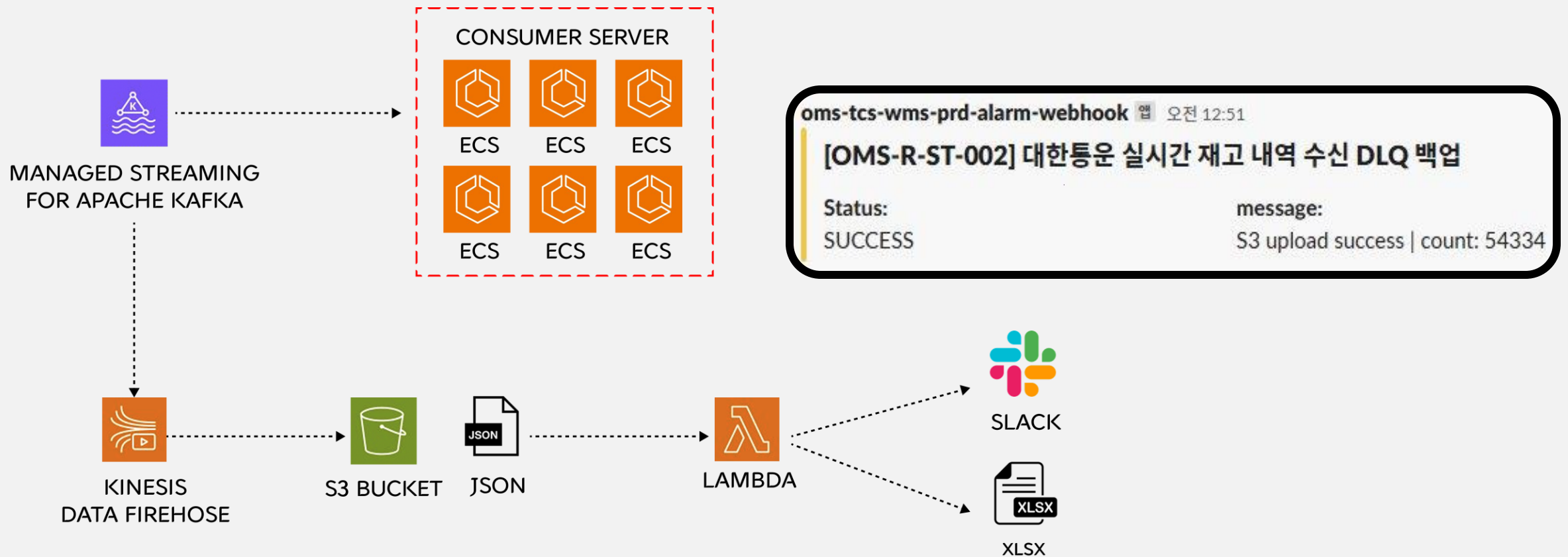
2-2. 입고-출고 주문 연동 개선 후

KAFKA 기반의 이벤트 스트리밍 구조로 전환하여 실시간성, 확장성, 신뢰성을 갖춘 연동 체계 구현



2-3. 실패한 메시지도 놓치지 않는 DLQ 백업 프로세스

KAFKA 기반 실시간 연동 중 예외 데이터를 DEAD LETTER QUEUE 적용 규칙으로 안전하게 수집 및 백업하고, 운영 대응까지 자동화



3. 실시간성과 확장성을 위한 데이터 연동 개선 효과

데이터 연동 속도뿐만 아니라 처리 방식, 장애 감지 및 복구까지 효과성 확인

데이터 연동 속도

개선 전

최대 3,147초



개선 후

최대 47초



우와, 67배 이상의 성능 향상이라니!
정말 실시간에 가깝게 데이터가 연동된 것을 알 수 있죠?

4. 데이터 연동 전환 과정에서 마주친 현실과 선택

우리가 마주한 현실 과제

우리가 선택한 해결 전략

운영 복잡도 증가

- 인프라와 운영의 복잡성
- Kafka 운영 경험 부족으로 초기 안정성 확보에 리스크 존재

- AWS MSK를 선택하여 초기 운영부담 완화
- 조직 내 플랫폼엔지니어링팀의 인프라 지원

기존 시스템과의 연계 부담

- 배치 위주의 기존 시스템
- 이벤트 기반 전환에 따라 일부 기존 시스템과의 아키텍처 조정 필요

- 단계적 전환
- 물류센터와 접목된 데이터 연동 프로세스에 한해 이벤트 구조로 운영

메세지 순서와 중복 처리

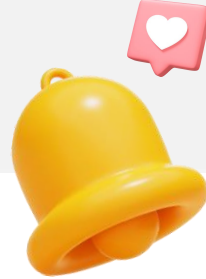
- 파티션 간 순서 보장을 위한 설계 필요 및 중복 가능성 고려

- 순서 보장이 필요한 메세지는 Key 기반 파티셔닝 적용
- 중복처리 방어로직 적용

개발자 및 운영자 학습 곡선

- Kafka 및 MSK를 경험한 구성원이 없음
- 도입 초기에 많은 학습이 필요함

- 도입 초기 PoC(Proof of Concept)와 파일럿 시스템 구축
- 운영 및 개발 가이드 제작 (표준화)



CHAPTER.2

실시간성과 확장성을 위한 시스템 모니터링

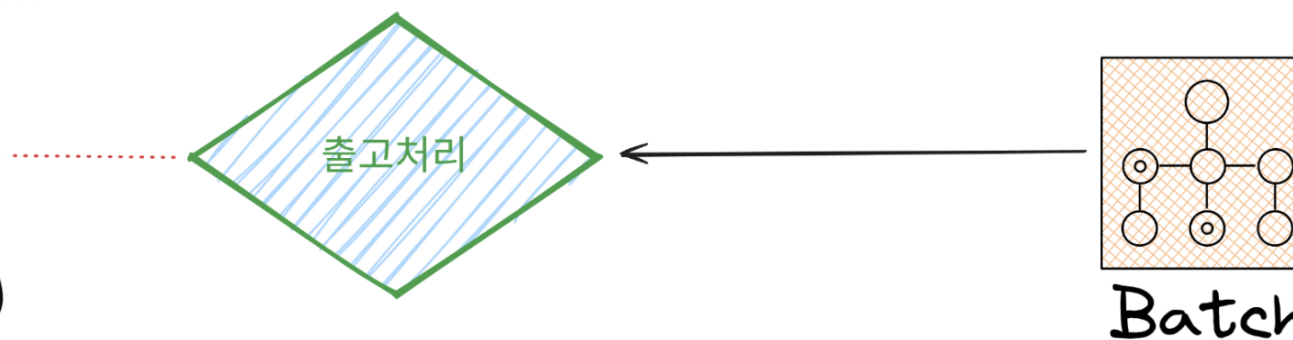
BUILDING OBSERVABILITY FOR REAL-TIME CONTROL AND SCALE

평화롭게 꿀잠을 자던 올리브영 인벤토리 서비스 개발자의 어느 밤

BATCH 위주의 시스템을 운영하던 시절 중 그 어느 날의 이야기



숙면을 취하고 있는 김올영님



평화롭게 꿀잠을 자던 올리브영 인벤토리 서비스 개발자의 어느 밤

갑자기 올리는 슬랙 메시지



이골드님_옴니 서비스 개발자 7월 7일 오전 8:50

안녕하세요, 금일자 매장 물류 입고 내역이 올리브원에서 확인되지 않고 있으며, 입고 대상 상품의 전자라벨은 솔드아웃으로 표기되어 유지되고 있습니다. 재고 배치가 완료되었는지 문의드립니다.



61개의 댓글



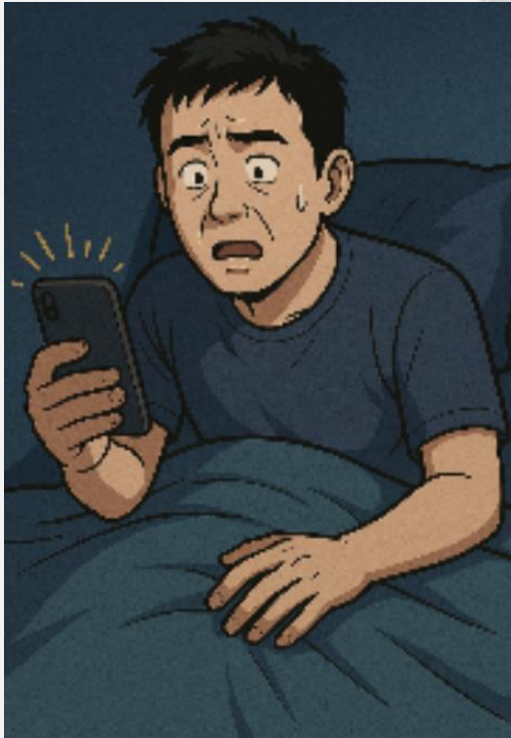
평화롭게 꿀잠을 자던 올리브영 인벤토리 개발자의 어느 날

판매 가능한 제품이 버젓이 있음에도 매장 내 스마트 전자라벨에 "품절(SOLD OUT)" 표시가 떠 있는 상태 확인



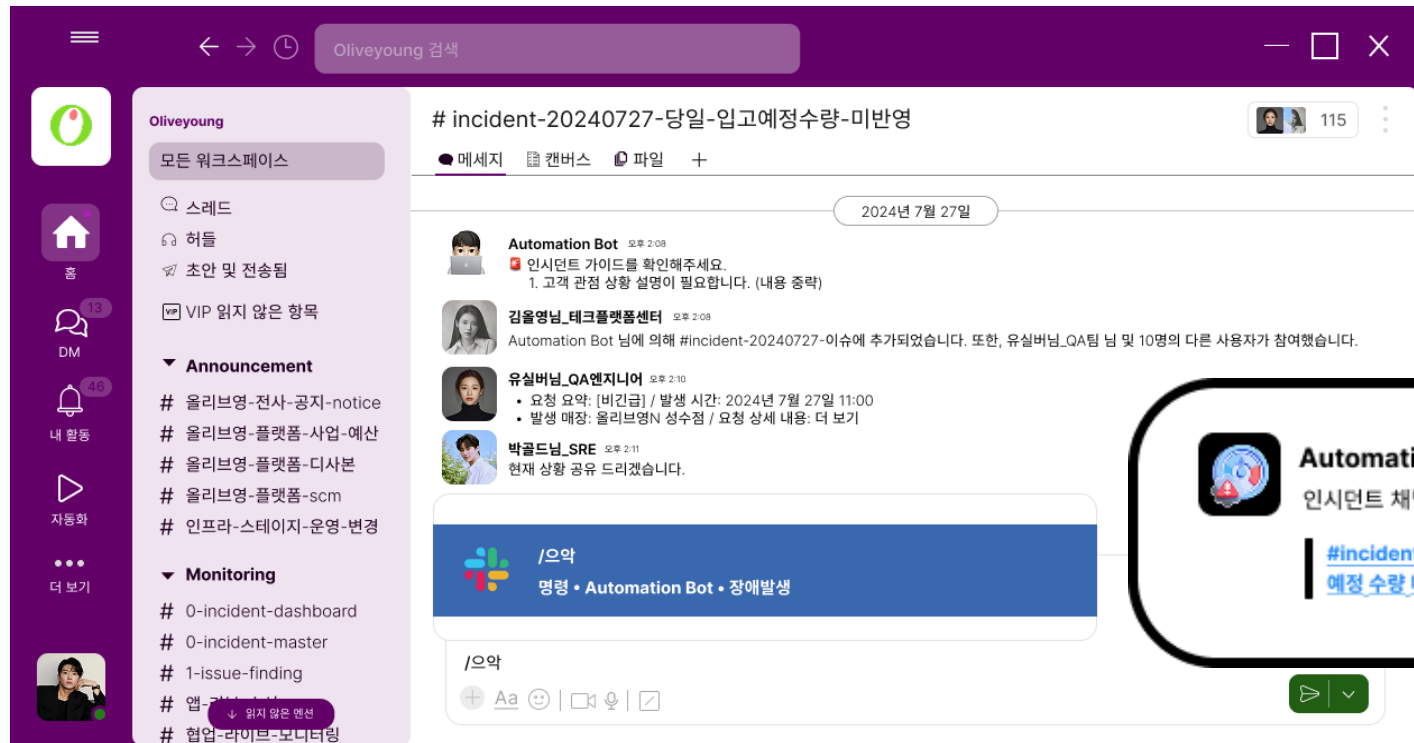
슬랙메시지를 확인 하고 원인을 파악하기 시작한 인벤토리 개발자

로그 확인 결과, 데이터 지연으로 인한 인시던트가 발생했음을 확인함



"/으악봇"을 호출해 인시던트를 선언하고 밤새 장애 보고서 작성

올리브영 개발자가 노트북을 토렘처럼 항상 가지고 다녀야 했던 나날들



Automation Bot 오후 1:05

인시던트 채널이 생성되었습니다.

[#Incident-20240727 출하 생성 매장 입고 미진행으로 인한 당일 입고 예정 수량 미반영](#)

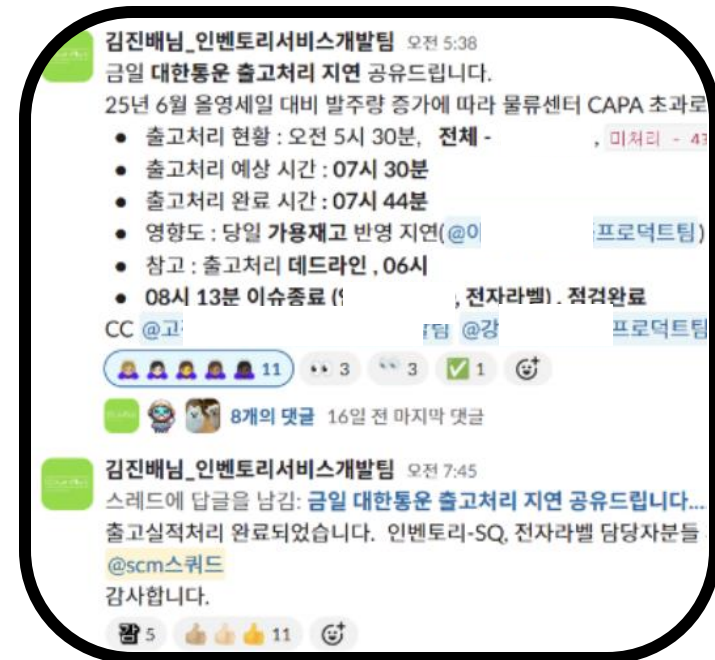
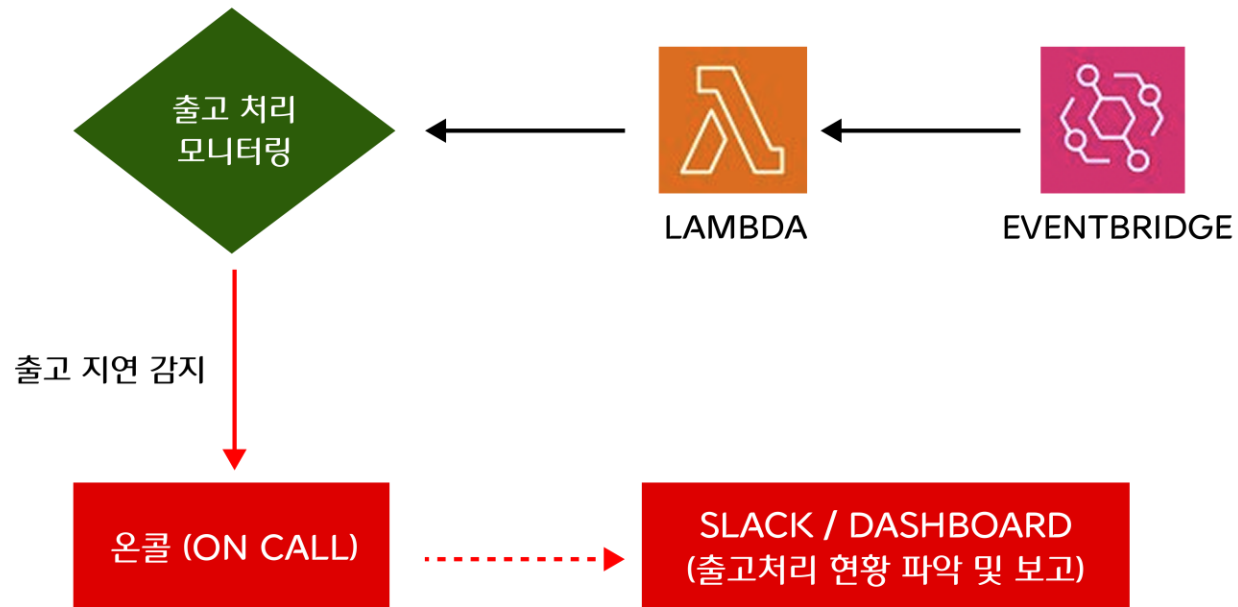
당장의 인시던트는 해결했지만, 재발 방지책이 필요했던 인벤토리 담당자들

인시던트 상황 예측 및 선제적 대응에 대한 필요성을 각인하고, 이에 대한 해결책 모색



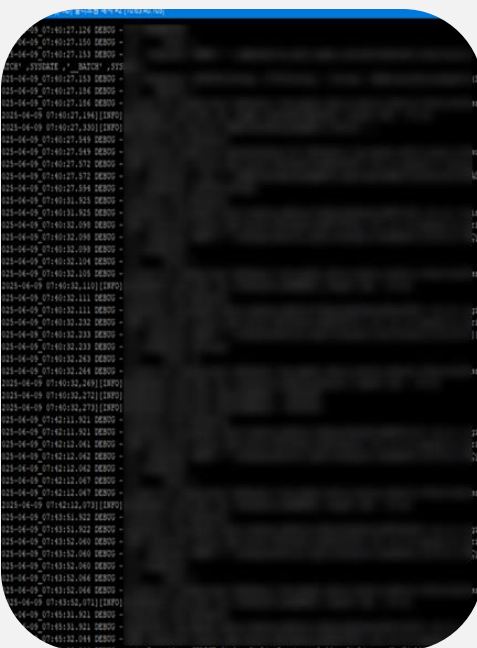
선제적 대응 체계와 가시성 확보를 위한 관제 체계 전환

사후 대응 체계에서 벗어나, 모든 시스템을 실시간 모니터링하고 이상 징후를 조기에 탐지하는 선제 대응 체계로 전환

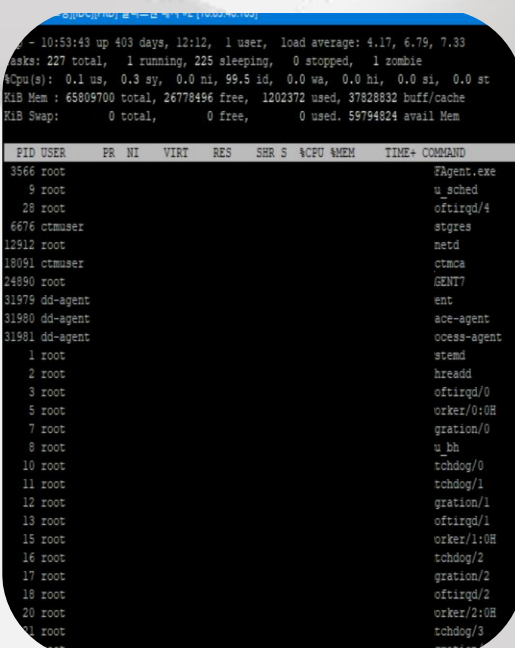


올리브영 개발자의 시선이 머무르는 대시보드 화면들

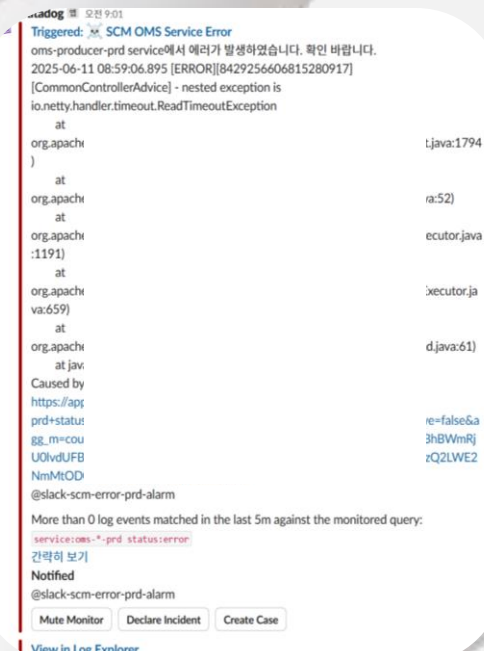
단순히 화면을 보는 것이 아니라, 데이터 흐름의 이상을 읽고 장애 조짐과 이상 징후 감지



데이터 센터 – Console Log



데이터 센터 – TOP, CPU/MEM



어플리케이션 로그/성능 모니터링



슬랙 에러 알람

올리브영 개발자의 시선이 머무르는 대시보드 화면들

단순히 화면을 보는 것이 아니라, 데이터 흐름의 이상을 읽고 장애 조짐과 이상 징후 감지



Datadog System ECS Dashboard

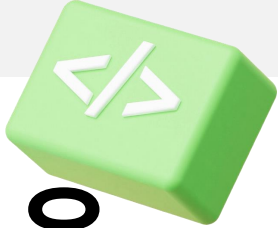
올리브영 개발자의 시선이 머무르는 대시보드 화면들

단순히 화면을 보는 것이 아니라, 데이터 흐름의 이상을 읽고 장애 조짐과 이상 징후 감지



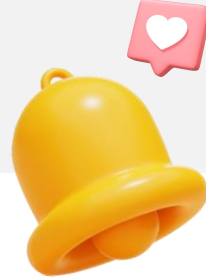
Datadog System ECS Dashboard

온디맨드 물류, 그 가능성의 시작은 시스템에서 진화하기 시작했습니다



올리브영 물류 시스템 개선은
단순한 '서비스'의 변화가 아닌
사용자와 현장의 '경험'을 바꾸는
여정이었습니다





CHAPTER.3

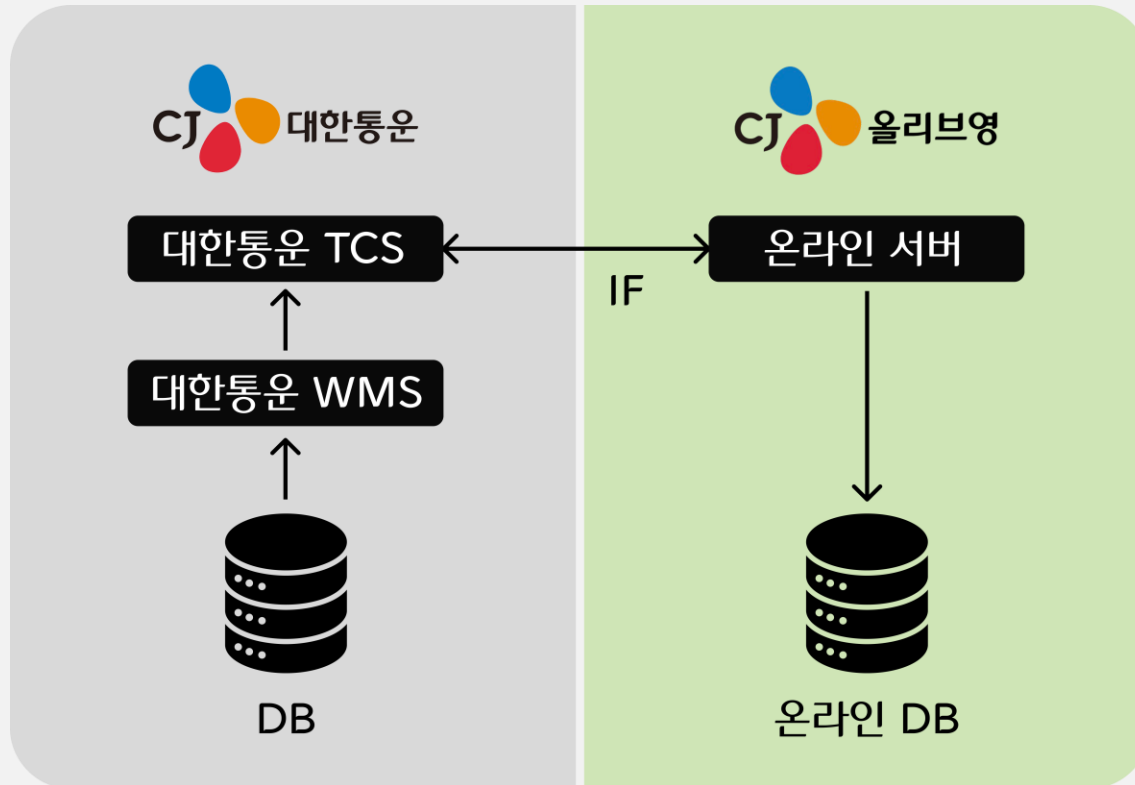
실시간성과 확장성을 위한 RDB 중심 구조의 탈피

MOVING BEYOND RDB-CENTRIC ARCHITECTURE

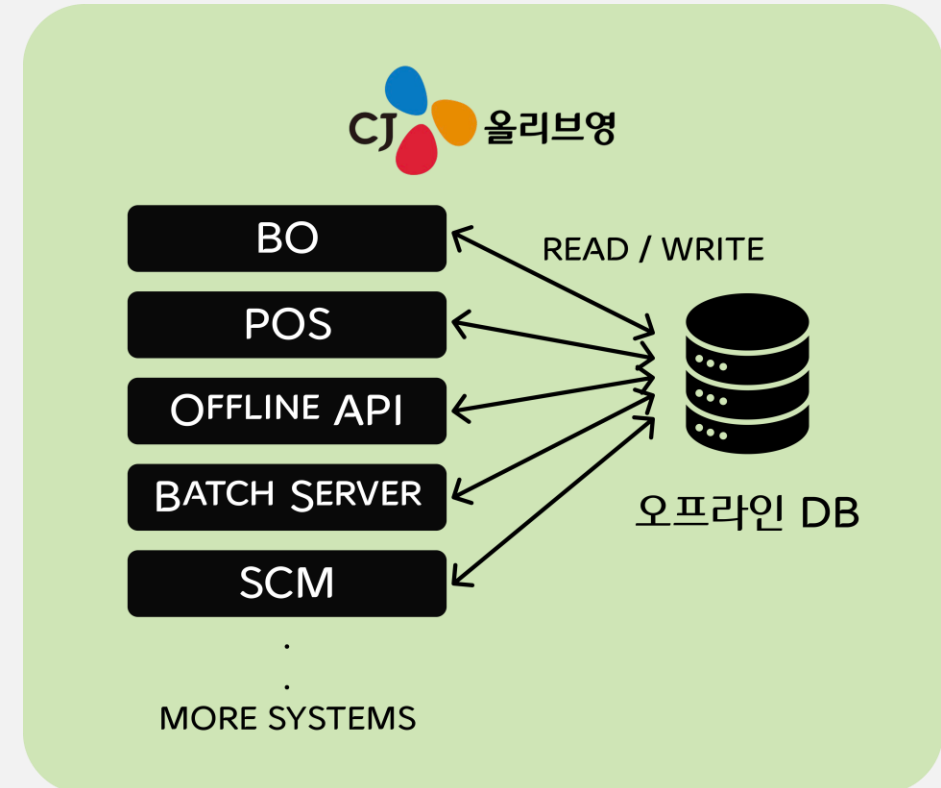
올리브영의 온·오프라인 재고 도메인 이해하기

옴니채널(OMNI-CHANNEL)로 구성된 올리브영은 온라인 재고와 오프라인 재고를 별도로 관리 중

온라인



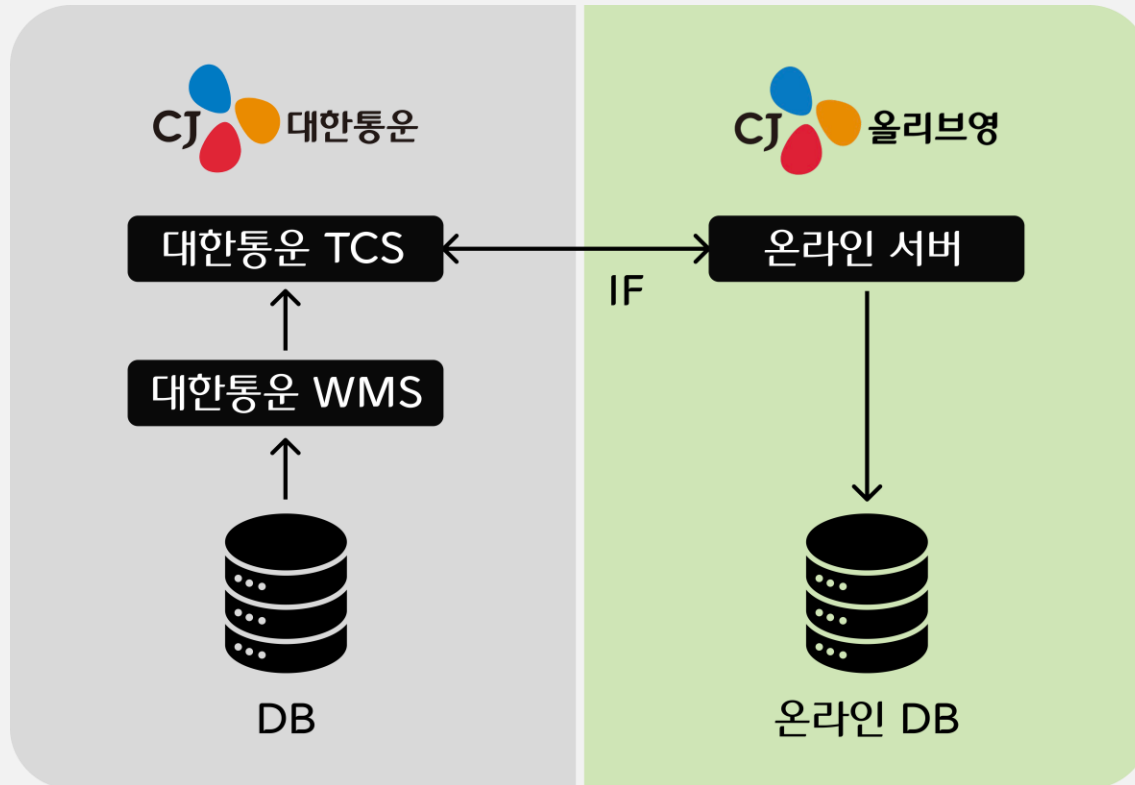
오프라인



올리브영의 온·오프라인 재고 도메인 이해하기

옴니채널(OMNI-CHANNEL)로 구성된 올리브영은 온라인 재고와 오프라인 재고가 별도로 관리되고 있음

온라인

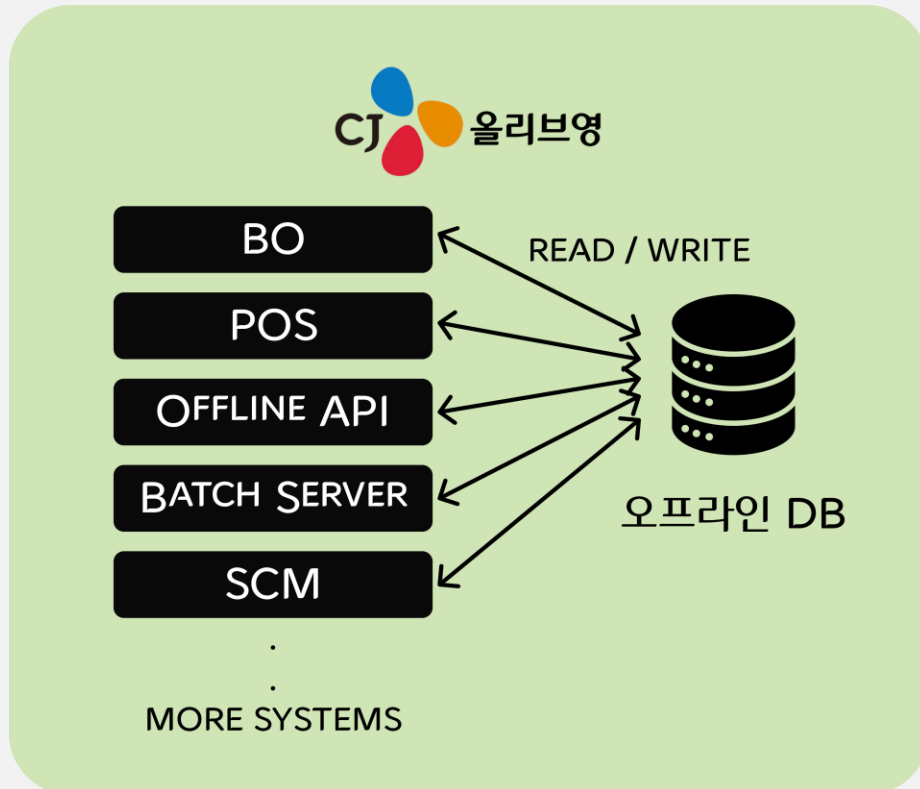


- 대한통운 물류센터의 재고
- 대한통운에서 변경되는 재고를 IF 로 싱크
- 상품 가짓수 20,000개
- 온라인몰 일반 주문시 사용

올리브영의 온·오프라인 재고 도메인 이해하기

옴니채널(OMNI-CHANNEL)로 구성된 올리브영은 온라인 재고와 오프라인 재고가 별도로 관리되고 있음

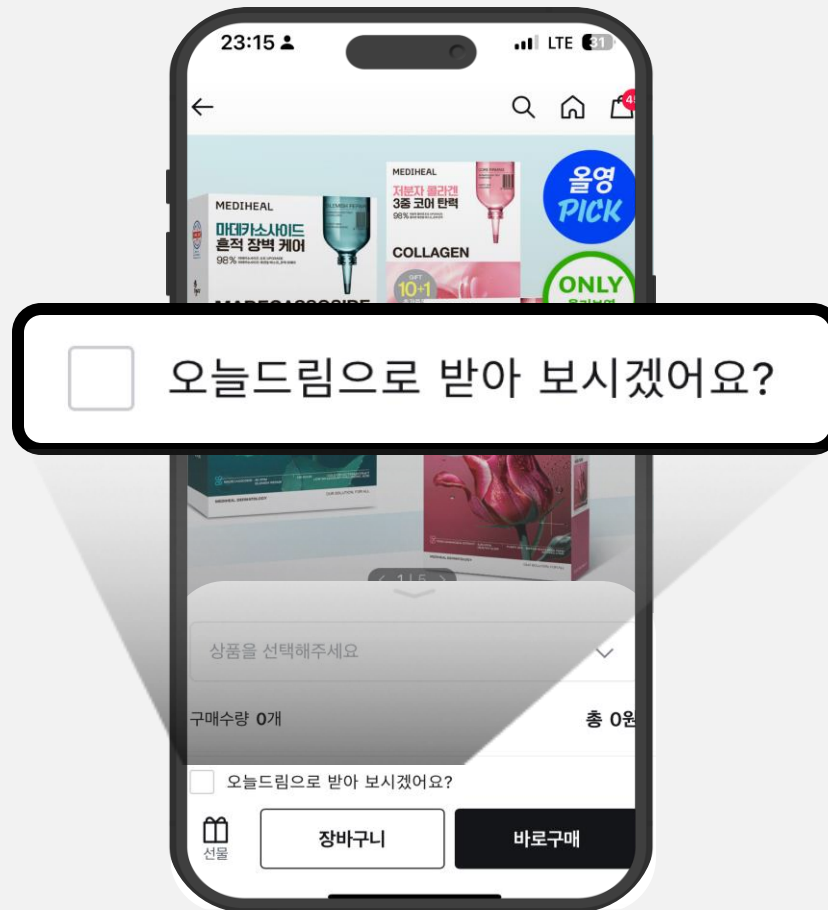
오프라인



- 올리브영 매장의 재고
- 자사 시스템에서 변경되는 재고를 오프라인DB에 직접 반영
- 상품 가짓수 1,100만개
- 오프라인매장 구매, 온라인몰 오늘드림 서비스에서 사용

"오늘드림" 서비스는 왜 오프라인 재고 조회 API를 사용할까?

옴니채널(OMNI-CHANNEL)로 구성된 올리브영이 온라인 채널에서 오프라인 재고 DB를 사용하는 이유



"오늘드림" 서비스는 왜 오프라인 재고 조회 API를 사용할까?

옴니채널(OMNI-CHANNEL)로 구성된 올리브영이 온라인 채널에서는 오프라인 재고 DB를 사용하는 이유



"오늘드림" 서비스란?

올리브영이 2018년 업계 최초로 도입한 O2O 서비스로, 온라인몰에서 "오늘드림" 옵션을 선택하면 고객의 주소지 인근 오프라인 매장에서 배달대행사를 통해 고객에게 당일배송하는 서비스입니다.



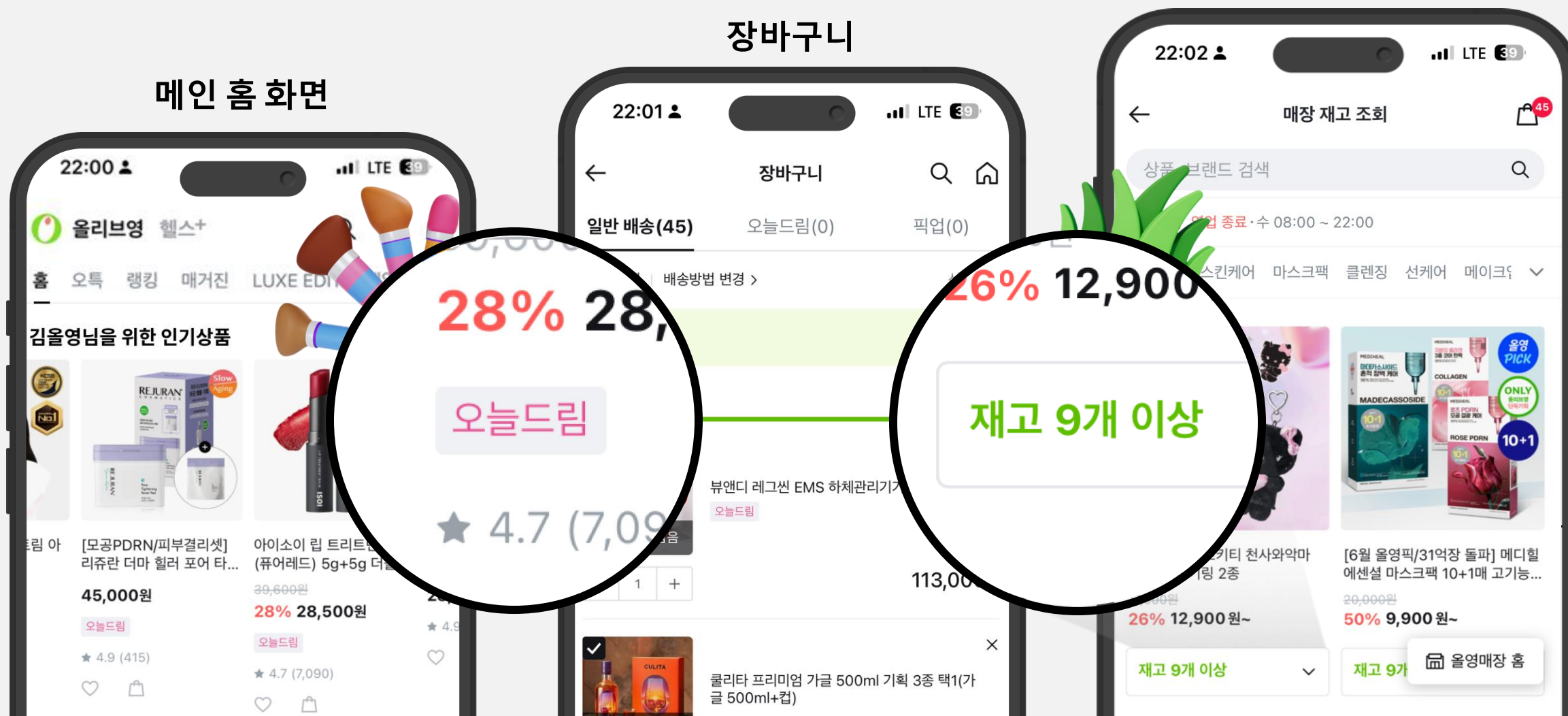
"오늘드림" 서비스는 왜 오프라인 재고 조회 API를 사용할까?

오늘드림 서비스 뿐만 아니라 매장 재고 조회 등 다양한 목적으로 오프라인 재고 조회

매장 재고 조회

장바구니

메인 홈 화면



2023년 6월 대규모 이벤트에서 발생한 전사 장애

올영세일 진행 중, 오프라인 DB를 사용하는 오늘드림에서 시작된 전사 시스템 장애



김올영님_플랫폼엔지니어링팀 오전 10:41

오늘드림 10시 40분경부터 오류가 발생 중입니다.



174개의 댓글



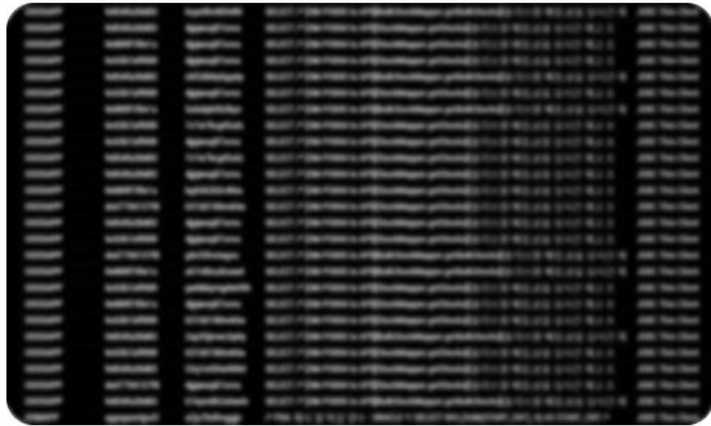
2023년 6월 대규모 이벤트에서 발생한 전사 장애

올영세일 진행 중, 오프라인 DB를 사용하는 오늘드림에서 시작된 전사 시스템 장애



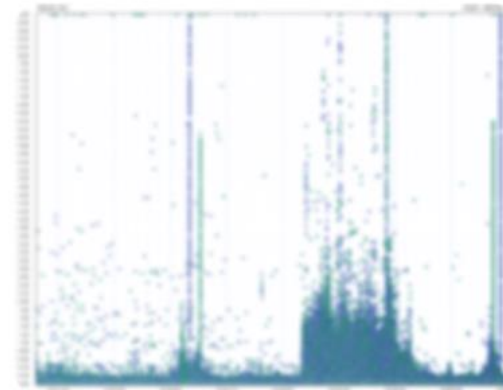
김진배님_인벤토리서비스개발자 오전 10:44

오늘드림 재고조회 쿼리가 엄청나네요.



김기술님_리테일서비스개발자 오전 10:56

DB 사용률에 따라 POS 중계 서버 쓰레드 응답 시간도 양상이 비슷해 보입니다.



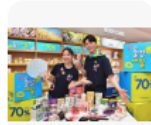
연일 기네스를 달성하는 과정에서 임계치에 도달한 오프라인 DB

매출은 신기록(기네스 기록)을 달성하고 있지만, 그만큼 부하가 커지면서 DB 부하에 대한 분석과 해결책 모색 필요

KDN 한국면세뉴스 · 2021.06.13.

CJ올리브영, 올영세일 매출 기네스 기록...1072억원 역대 최대 매출

CJ올리브영이 여름 '올영세일'에서 코로나 이전 매출을 뛰어넘는 사상 최대 매출을 기록했다고 13일 밝혔다. 행사기간 7일간 1072억원 매출(취급고 기준) 기네스를 기록한 것이다. 이는 지난봄 세일(+30%)은 물론, 코로나19 이전인 2019년 여름 세일...



녹색경제신문 · 2023.09.10.

[유통가 레이더] 올리브영, 가을 '올영세일' 매출 28% 증가... "외국인과..."

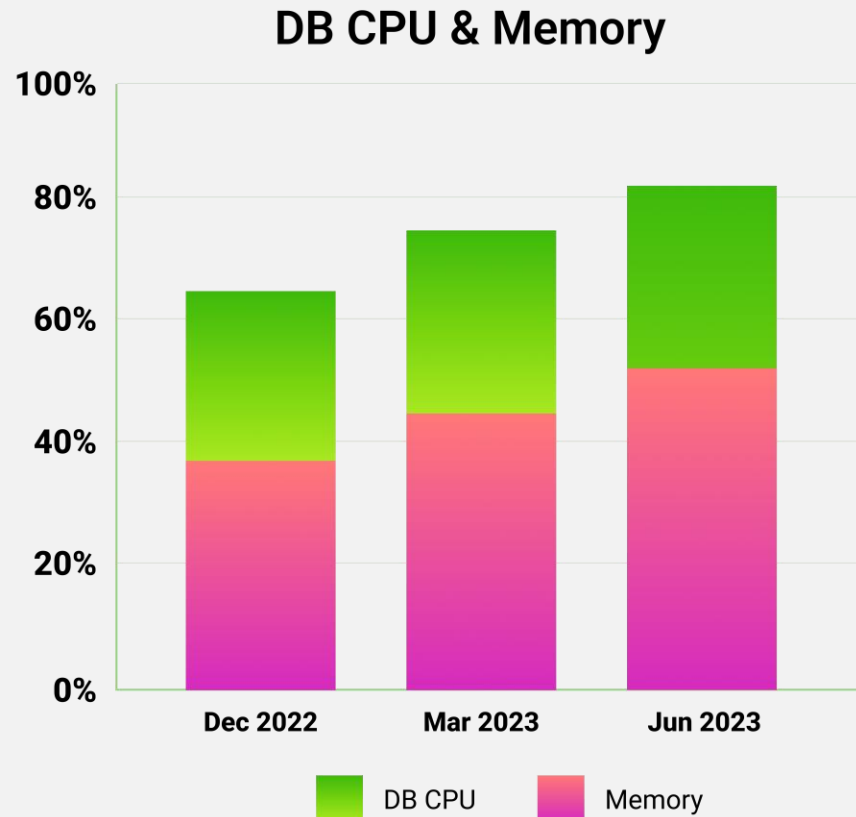
CJ올리브영(이하 올리브영)이 지난달 31일부터 9월 6일까지 진행한 '올영세일' 매출을 분석한 결과, 외국인과 온라인 매출이 큰 폭으로 증가하며 전년 대비 28% 늘었다고 10일 밝혔다. 이번 세일에서는 외국인 매출 증가가 눈에 띈다. 최근 방한 관광 정...



- 왜 자꾸 장사가 잘되는데!!?

연일 기네스를 달성하는 과정에서 임계치에 도달한 오프라인 DB

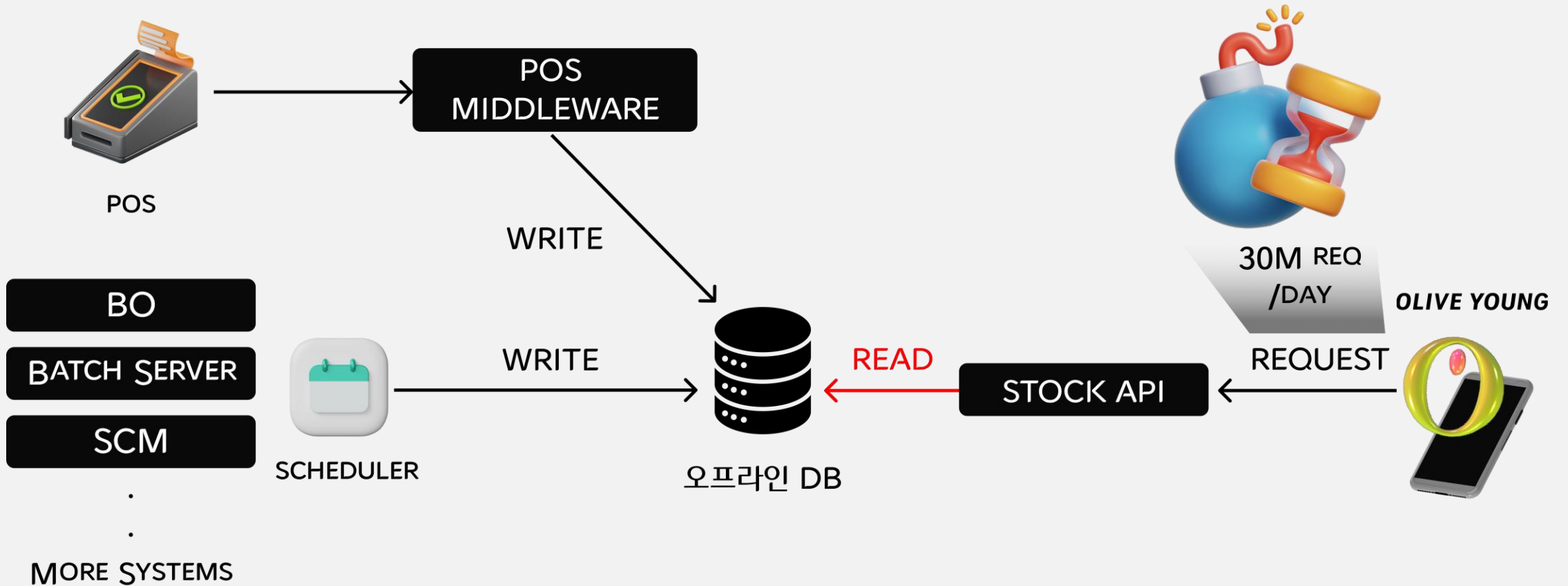
매출은 신기록(기네스 기록)을 달성하고 있지만, 그만큼 부하가 커지면서 DB 부하에 대한 분석과 해결책 모색 필요



- 정기 대규모 이벤트(올영세일)마다 매출 신기록(기네스) 달성
- 증가하는 트래픽과 주문량에 따라 DB 부하도 급증
- CPU 및 메모리 사용률이 지속적으로 고점 유지
- RDB 기반 구조에서 성능 확장이 한계에 도달
- 단순 증설이 아닌, 구조적 진단과 개선 필요

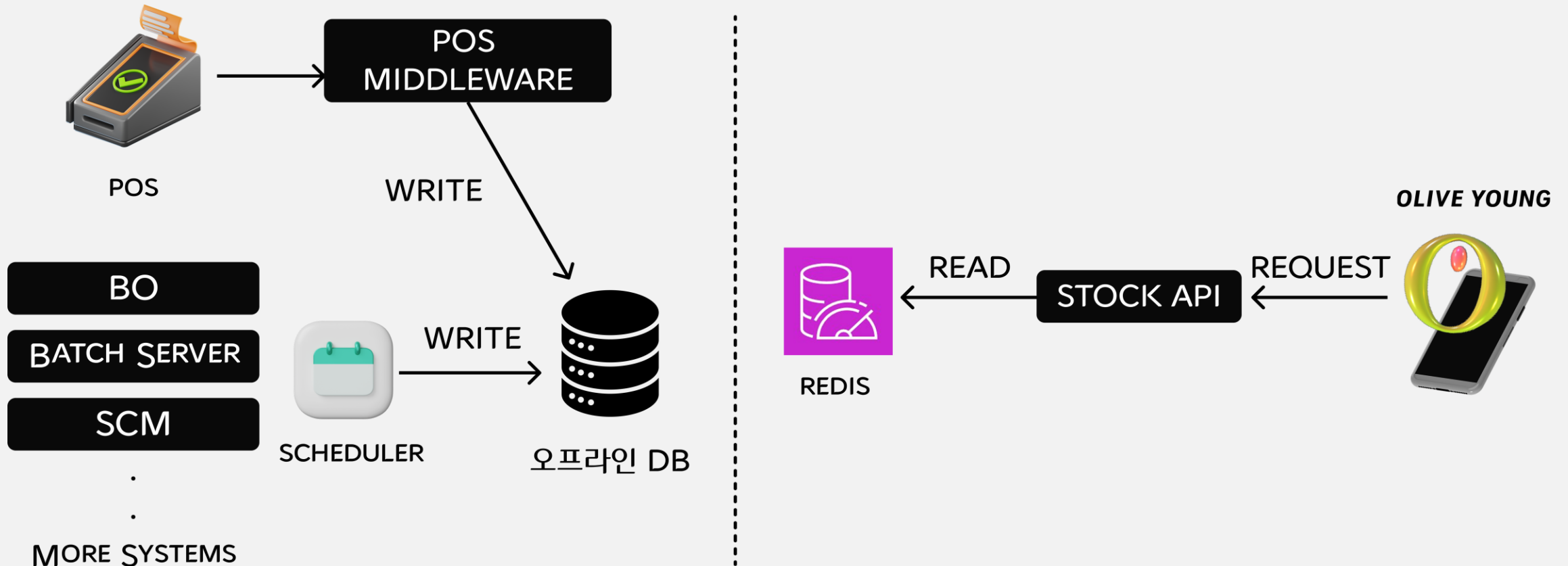
오프라인 DB 부하의 직접적 원인으로 지목된 오프라인 재고조회 API

올영세일 기간 일평균 3,000만번의 오프라인 DB 조회 발생

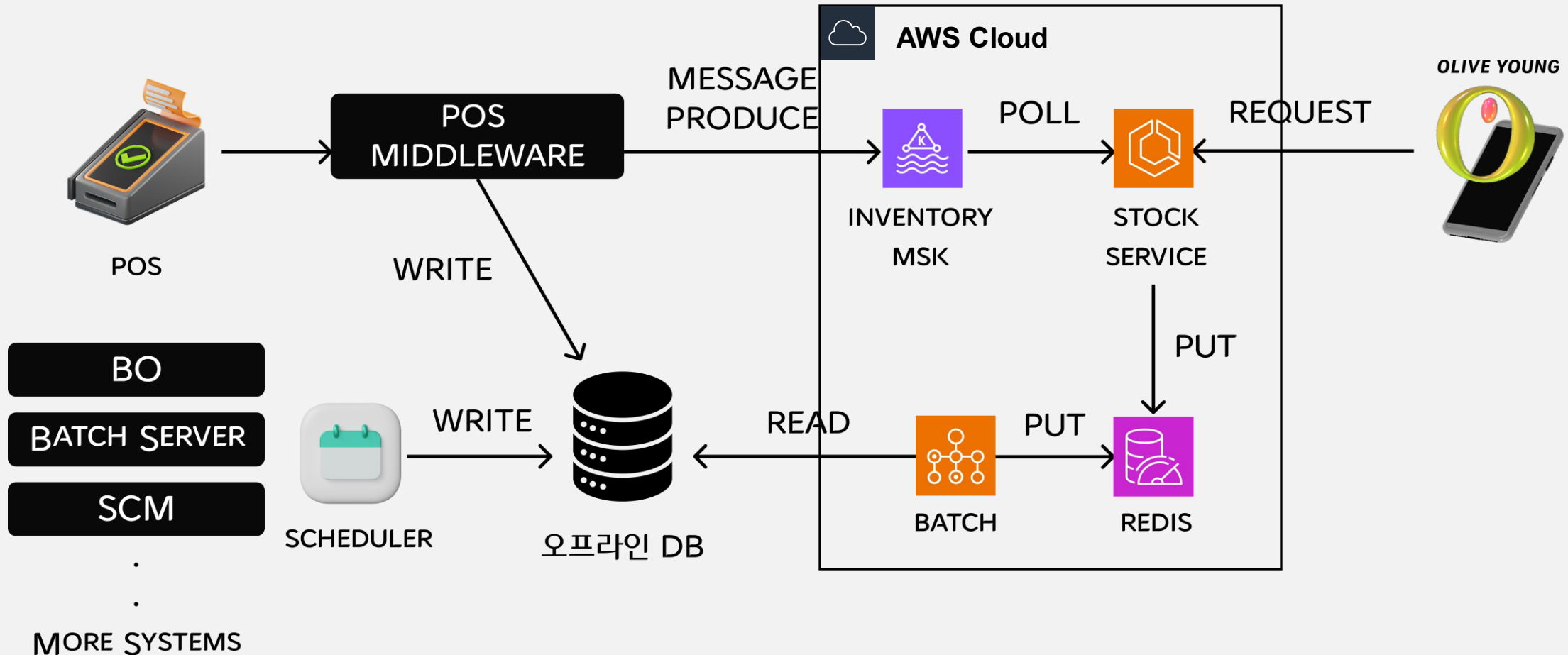


오프라인 DB 부하를 줄이기 위한 READ DB 분리

올리브영 온라인몰의 오프라인 재고 조회는 REDIS에서 조회하도록 분리 조치

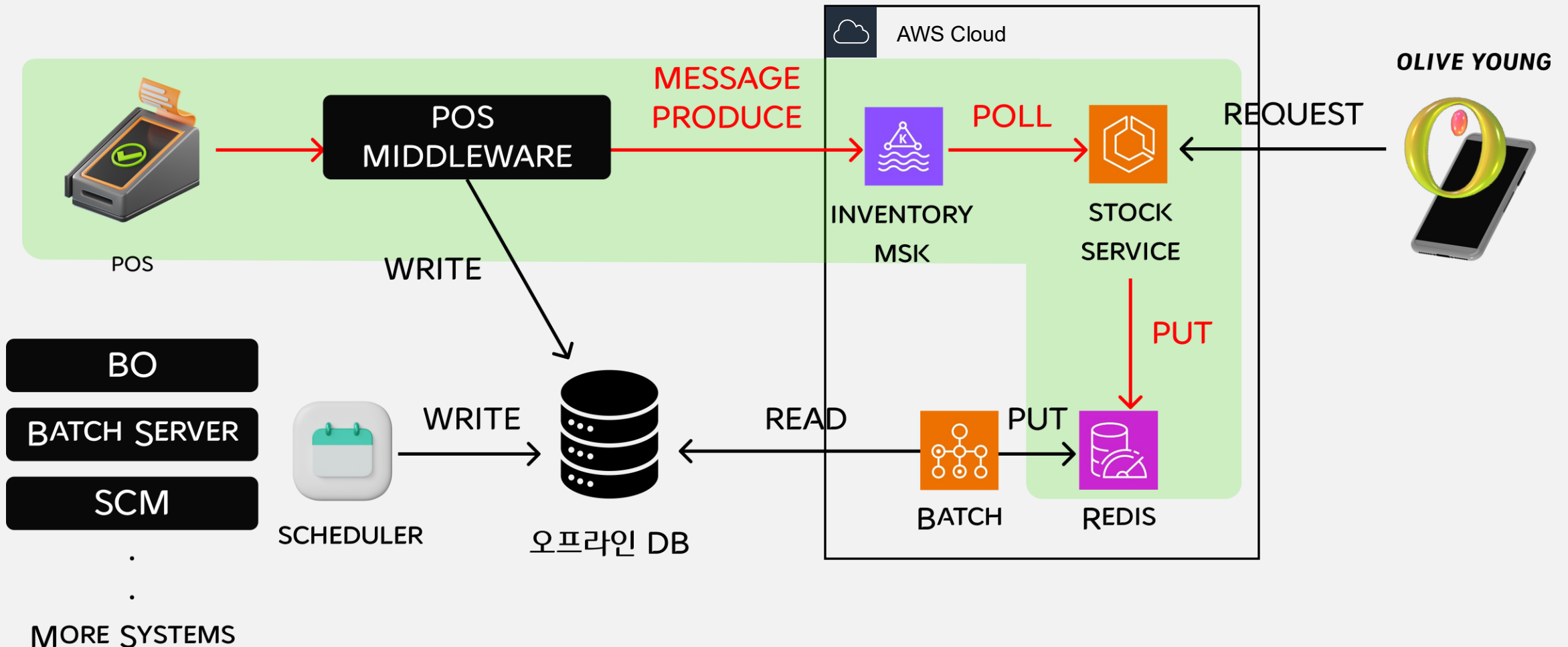


DB 분리 후 오프라인 재고 시스템 설계도



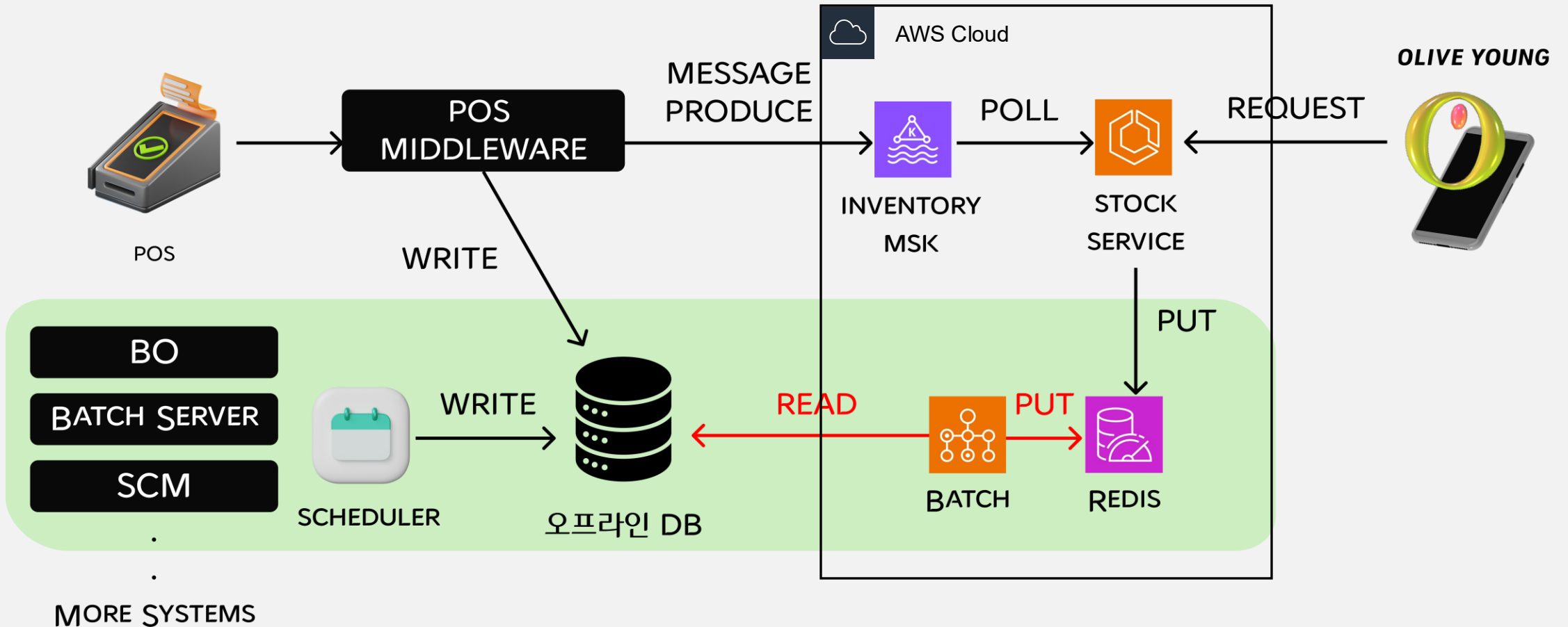
NEW 설계도: ORACLE ◇ REDIS 데이터 싱크

POS와 같은 실시간성 재고 변경은 ORACLE DB에 WRITE하는 시점에 KAFKA 메시지를 통해 REDIS에 재고 변경분 반영



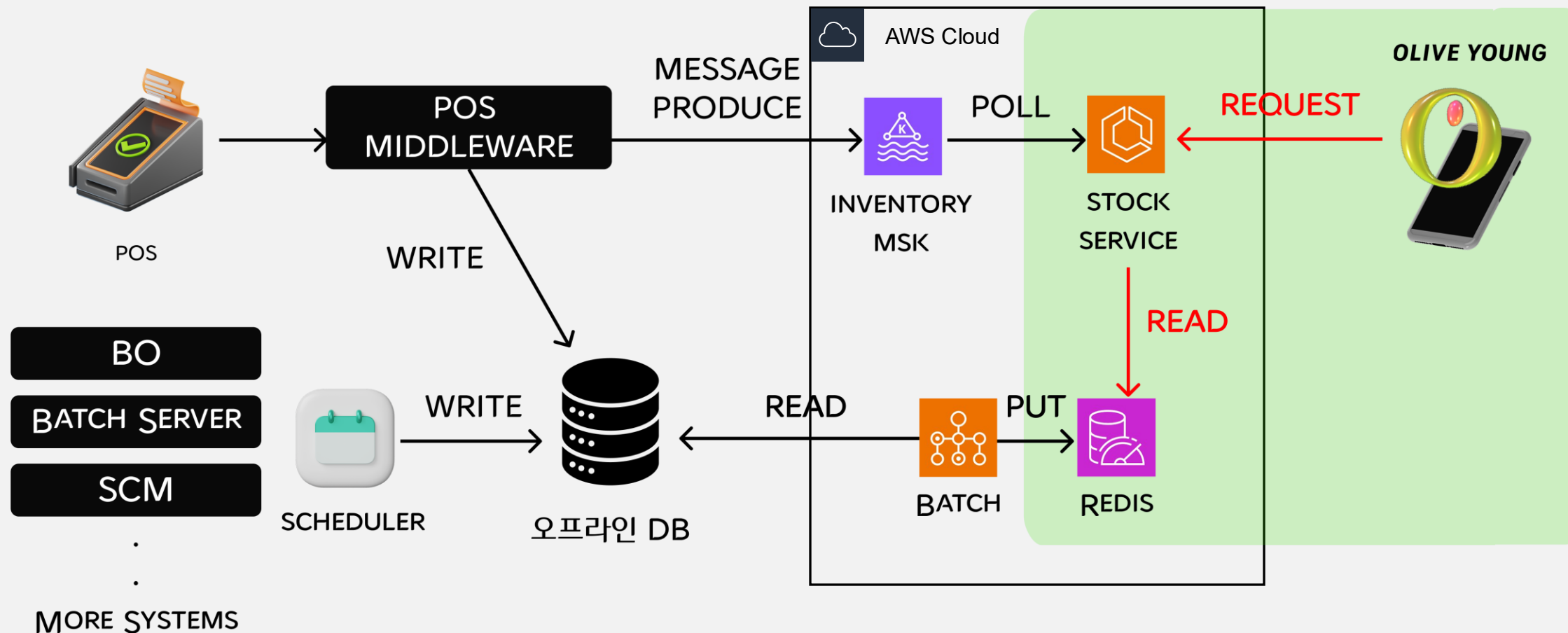
NEW 설계도: ORACLE ◇ REDIS 데이터 싱크

실시간성 재고 변경이 아닌 건은 BATCH로 ORACLE DB의 변경된재고 데이터를 주기적으로 읽어 REDIS 반영

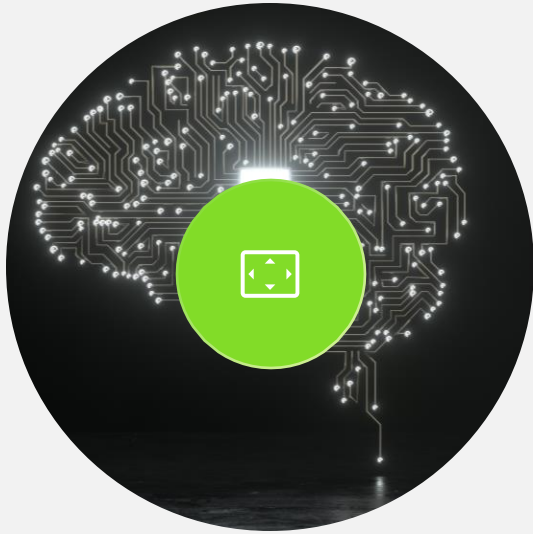


NEW 설계도: 조회 API의 조회 대상 DB 변경

조회 API는 Redis에서 재고 데이터 조회



FROM ORACLE TO REDIS: 실시간성을 얻고자 마주한 세 가지 이슈



동시성 제어

Row Lock이 없는 Redis 환경에서,
동시성은 어떻게 보장할 수 있을까?



조회 성능

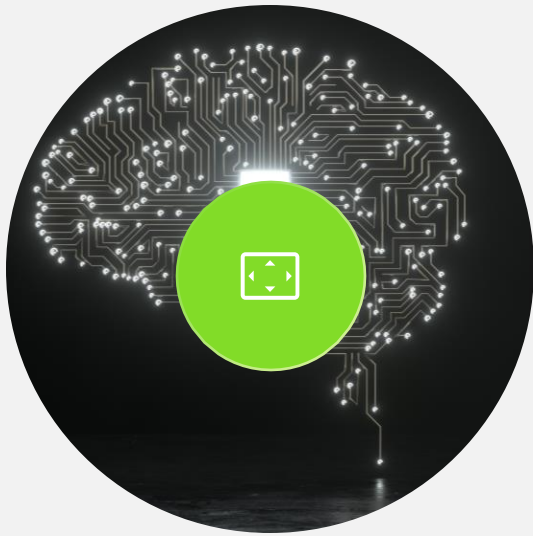
Index가 없는 Redis에서, 어떻게 특정
매장의 수천 개 상품 재고를 빠르게
조회할 수 있을까?



안정성

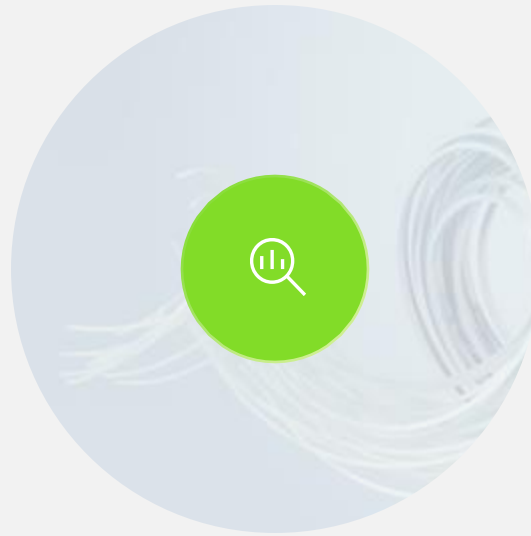
Redis 클러스터 장애상황에서 어떻게
안정성을 보장할 것인가?

① 동시성 제어 이슈



동시성 제어

Row Lock이 없는 Redis 환경에서,
동시성은 어떻게 보장할 수 있을까?



조회 성능

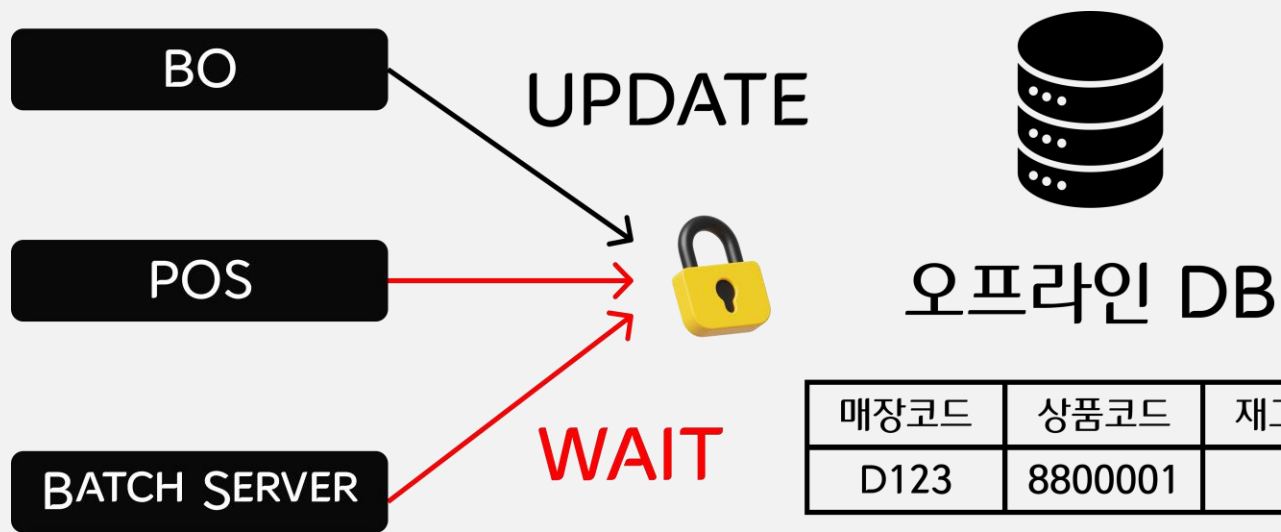
Index가 없는 Redis에서, 어떻게
특정 매장의 수천 개 상품 재고를
빠르게 조회할 수 있을까?



안정성

Redis 클러스터 장애상황에서 어떻게 안정성을
보장할 것인가?

① 동시성 제어 이슈: ORACLE VS REDIS



ORACLE

- 여러 클라이언트에서 동일 재고 업데이트 시, row-level lock으로 동시성 제어 가능

① 동시성 제어 이슈: ORACLE VS REDIS



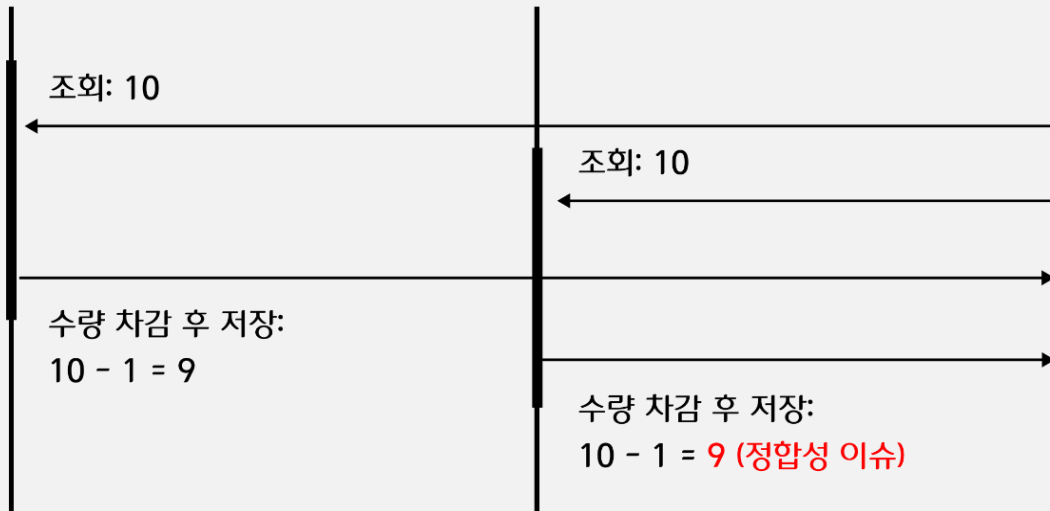
BATCH



STOCK SERVICE



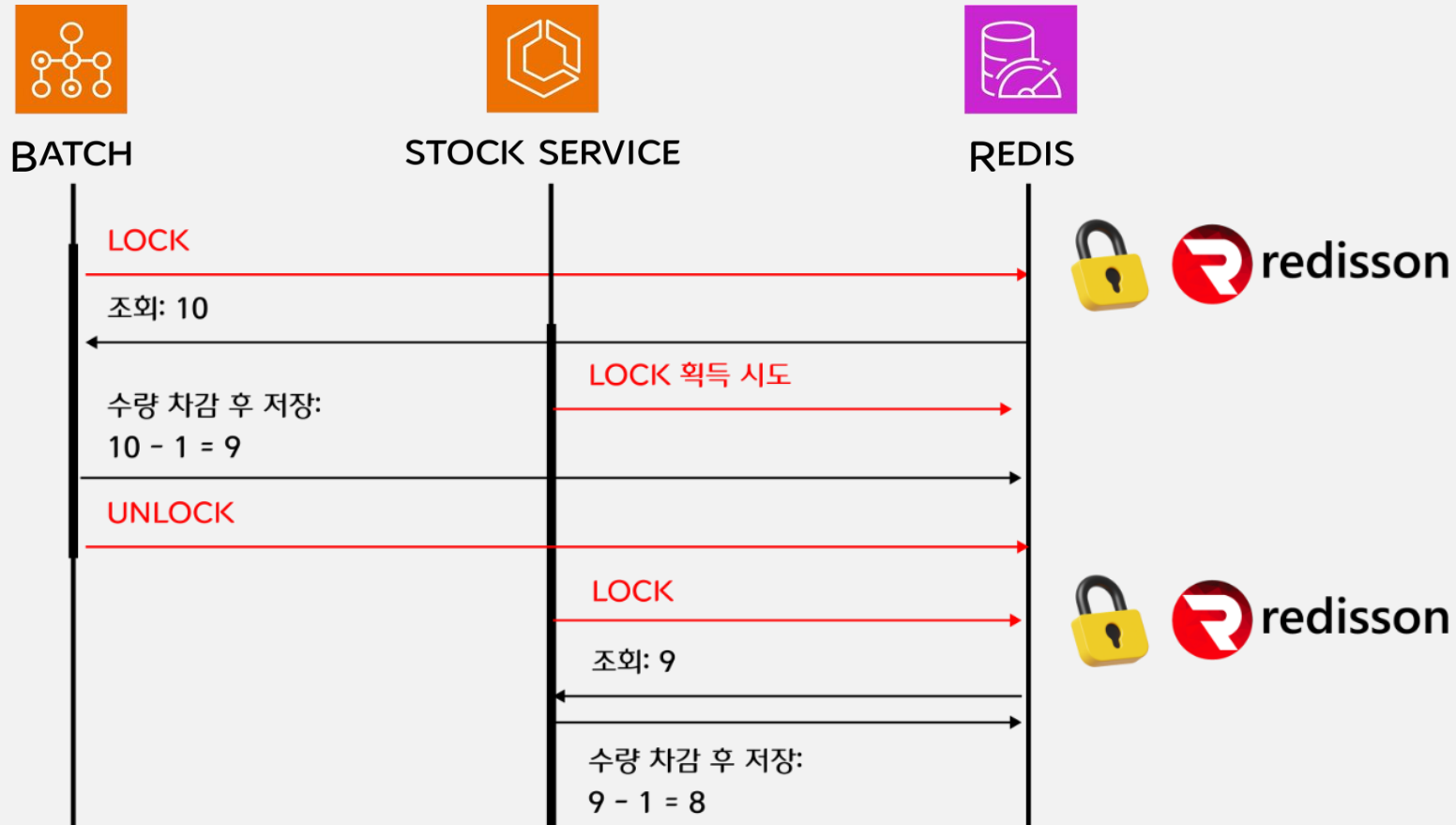
REDIS



REDIS

- 기본적으로 단일 스레드로 동작해 단일 명령어에서는 충돌 없음
- 하지만 GET → 계산 → SET처럼 복합 명령어가 수행되는 경우, 중간에 다른 클라이언트가 개입할 수 있어 정합성 이슈 발생
- 별도 동시성 제어 로직 없이는 경합 발생 가능성 존재

① 동시성 제어 이슈: ORACLE VS REDIS



REDIS + REDISSON

- Redisson은 분산락 기능을 제공하여 멀티 클라이언트 환경에서도 재고 동기화 보장
- Reactive 방식을 지원하여 조회 성능 극대화
- 재고 조회 및 갱신 시점에 Lock 획득 → 작업 수행 → Lock 해제로 안정성 확보

```
11 @Target(AnnotationTarget.FUNCTION)
12 @Retention(AnnotationRetention.RUNTIME)
13 Annotation class Lockable(
14     Val Key: string
15 )
```

커스텀 어노테이션 생성

```
35 @Lockable(key = "#stockKey")
36 Override fun order(
37     StockKey: StockKey,
38     OrderQuantity: Long,
39 ): Stock {
40     Return findStockAndUpdate(
41         StockKey,
42         Stock.orderProperties
43     ) {
```

임계 영역이 필요한 곳에
어노테이션 붙이기

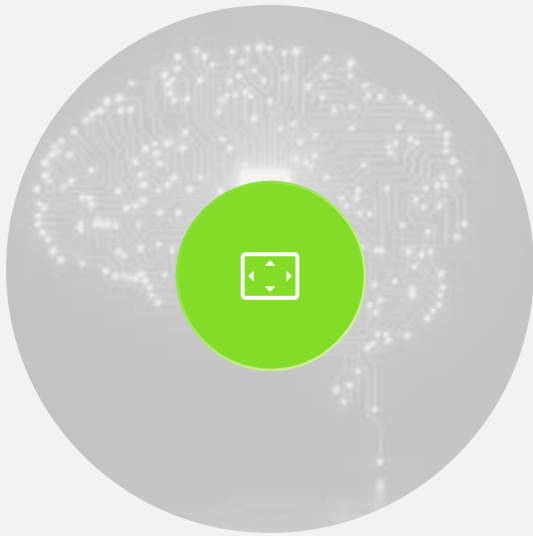
Key 기준으로 lock 생성

```
107 @Around(value = "@annotation(Lockable) && args(stockKey,..)")
108 Fun lockTheResource(proceedingJoinPoint: ProceedingJoinPoint, stockKey): Any? {
109
110     val lock = memoryDBConnectionFactory.getLock(LOCK_PREFIX + stockKey.getKey() + LOCK_SUFFIX)
111
112     return try {
113         if (!lock.tryLock(CLUSTER_WAIT_TIME, CLUSTER_LEASE_TIME, TimeUnit.MILLISECONDS)) {
114             throw IllegalStateException("락 획득 실패: ${stockKey.getKey()}")
115         }
116         ProceedingJoinPoint.proceed()
117     } catch (e: Exception) {
118         throw e
119     } finally {
120         if (lock.isLocked && lock.isHeldByThread(Thread.currentThread().id)) {
121             lock.unlock()
122         }
123     }
124 }
```

Redis get + set 전에 lock

Redis get + set 이후 unlock

② 조회 성능 이슈



동시성 제어

Row Lock 이 없는 Redis
환경에서, 동시성은 어떻게
보장할 수 있을까?



조회 성능

Index가 없는 Redis에서, 어떻게
특정 매장의 수천 개 상품
재고를 빠르게 조회할 수
있을까?



안정성

Redis 클러스터
장애 상황에서 어떻게
안정성을 보장할 것인가?

② 조회 성능 이슈: ORACLE VS REDIS

MEMORY DB는 단건 조회에서 DISK 기반 ORACLE보다 성능상 우위

EX) 매장 상품 재고 API : 특정 매장 특정 상품의 재고 조회

ORACLE

매장코드	상품코드	재고수량
D123	8800001	1

- 디스크 기반 관계형 DB
- 인덱스 탐색 → 테이블 로우 접근
- 응답 속도: 밀리초(ms) 단위

REDIS

```
Stock Hash Key
"{매장코드}:{상품코드}": {
  "quantity": 10,
  "productName": "상품명"
}
```

- 메모리 기반 Key-Value Store
- 키 기반 메모리 조회로 빠른 응답
- 응답 속도: 마이크로초(μ s) 단위

② 조회 성능 이슈: ORACLE VS REDIS

복수건 조회는 **SCAN** 기반 패턴 탐색으로 인해 **ORACLE** 대비 성능상 불리
 EX) 매장 전체 재고 API : 특정 매장의 모든 상품 재고 조회

ORACLE

매장코드	상품코드	재고수량
D123	8800001	1
	8800002	3
	8800003	4

인덱스를 활용해 매장 전체 재고 조회가능

INDEX : STORE_ID

REDIS

```
Stock Hash Key
"{매장코드}:{상품코드}": {
  "quantity": 10,
  "productName": "상품명"
}
```

1천만개가 넘는 전체 키를 패턴기반 SCAN 으로 반복 조회

```
SCAN 0 MATCH 매장코드:* 100
SCAN 커서 MATCH 매장코드:* 100
SCAN 커서 MATCH 매장코드:* 100
...
```

ORACLE 대신 REDIS 사용시 발생하는 이슈와 해결 : 조회 성능 이슈

OLIVE YOUNG

조회용도별로 성능을 최적화 하기 위해 REDIS 데이터셋을 3개로 구성

Stock Hash Key
"{매장코드}:{상품코드}": {
 "quantity": 10,
 "productName": "상품명"
}

특정 매장코드+상품 코드를 기준으로
재고를 조회할 때 사용

HASH KEY 기준 단건 조회

HGET 매장코드:상품코드 quantity

Store Hash Key
"store:{매장코드}": {
 "상품코드": 10,
 "상품코드": 3,
 "상품코드": 5
 ...
}

특정 매장코드를 기준으로
매장의 모든 상품재고를 조회할 때 사용

매장코드 HASH KEY 를 기준으로
필드를 HSCAN

HSCAN store:매장코드 0 COUNT 100
HSCAN store:매장코드 커서 COUNT 100
...

Product Hash Key
"product:{상품코드}": {
 "매장코드": 10,
 "매장코드": 5,
 "매장코드": 6,
 ...
}

특정 상품코드를 기준으로
상품코드를 가지고 있는
모든 매장의 상품재고를 조회할 때 사용

상품코드 HASH KEY 를 기준으로
필드를 HSCAN

HSCAN product:상품코드 0 COUNT 100
HSCAN product:상품코드 커서 COUNT 100
...

ORACLE 대신 REDIS 사용시 발생하는 이슈와 해결 : 조회 성능 이슈

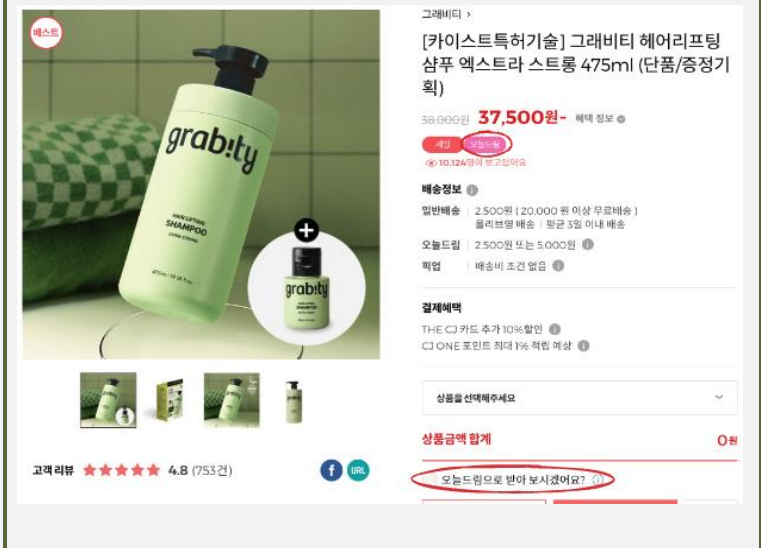
OLIVE YOUNG

조회용도별로 성능을 최적화 하기 위해 REDIS 데이터셋을 3개로 구성

Stock Hash Key

"{매장코드}:{상품코드}": {
 "quantity": 10,
 "productName": "상품명"
}

상품상세, 장바구니 등에서 사용



Store Hash Key

"store:{매장코드}": {
 "상품코드": 10,
 "상품코드": 3,
 "상품코드": 5
 ...
}

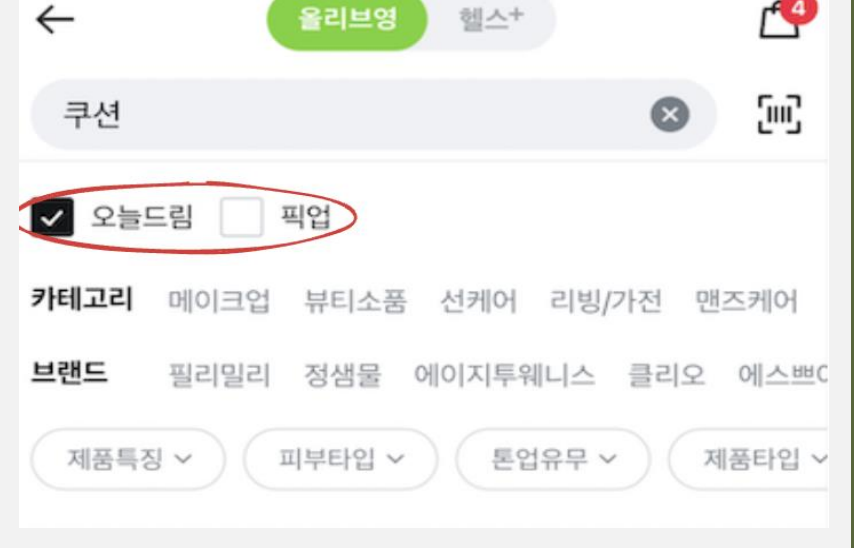
전자라벨 등에서 사용



Product Hash Key

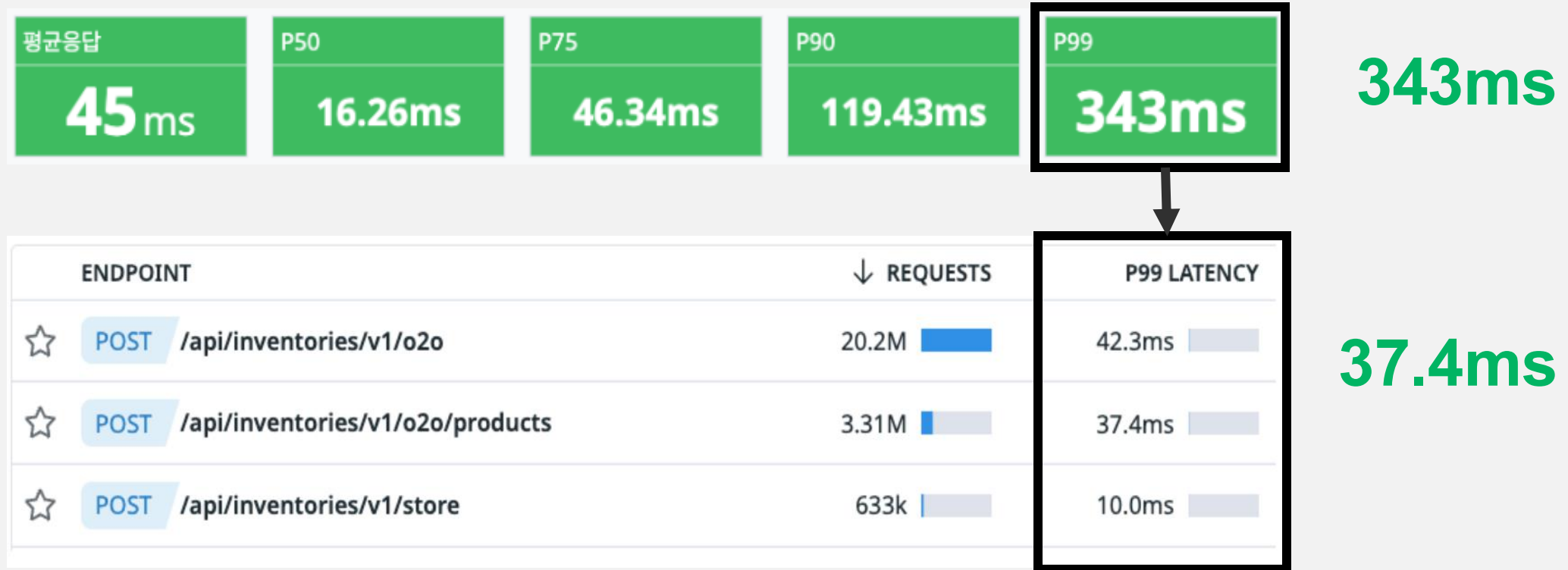
"product:{상품코드}": {
 "매장코드": 10,
 "매장코드": 5,
 "매장코드": 6,
 ...
}

검색엔진 등에서 사용



② 조회 성능 이슈: 개선 후 재고 조회 API 성능 약 10배 향상

ORACLE 기반 오프라인 재고 API 성능이 P99 기준으로 343MS → 37MS 향상

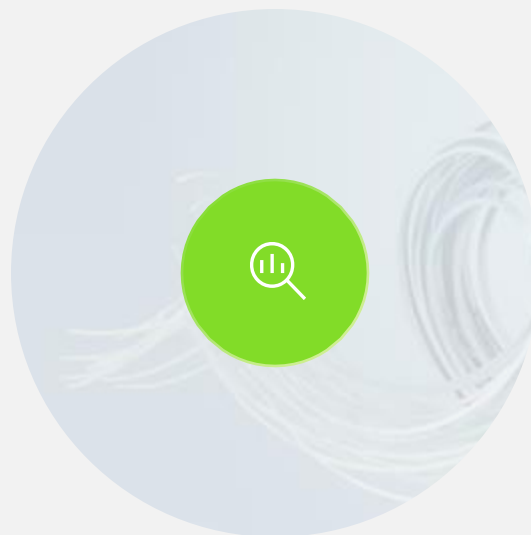


③ 안정성 이슈



동시성 제어

Row Lock 이 없는 Redis 환경에서, 동시성은 어떻게 보장할 수 있을까?



조회 성능

Index 가 없는 Redis 에서, 어떻게 특정 매장의 수천 개 상품 재고를 빠르게 조회할 수 있을까?

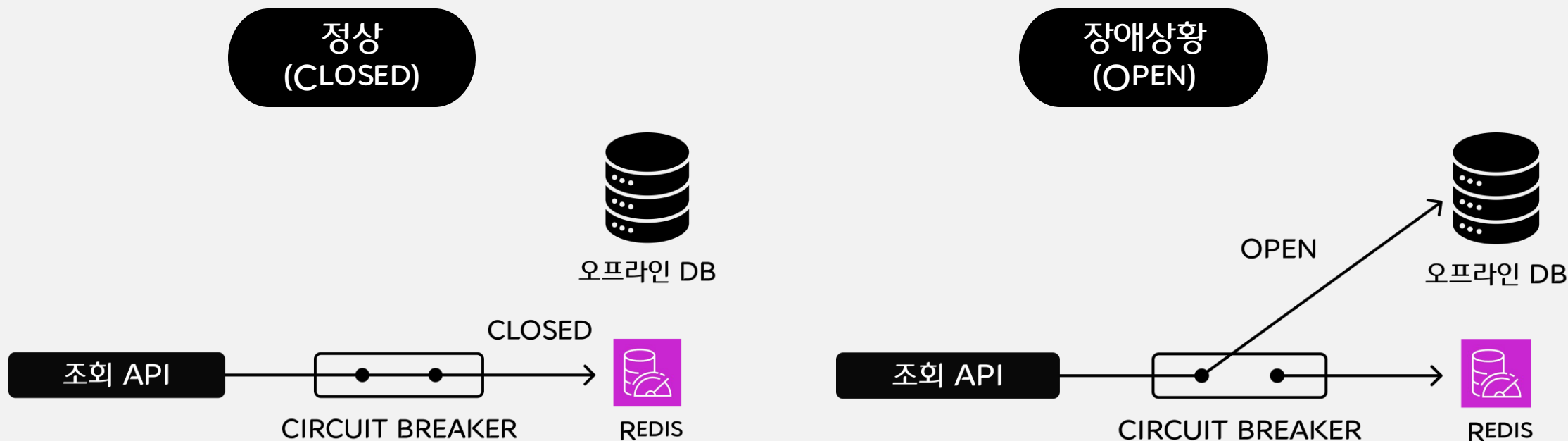


안정성

Redis 클러스터 장애 상황에서 어떻게 안정성을 보장할 것인가?

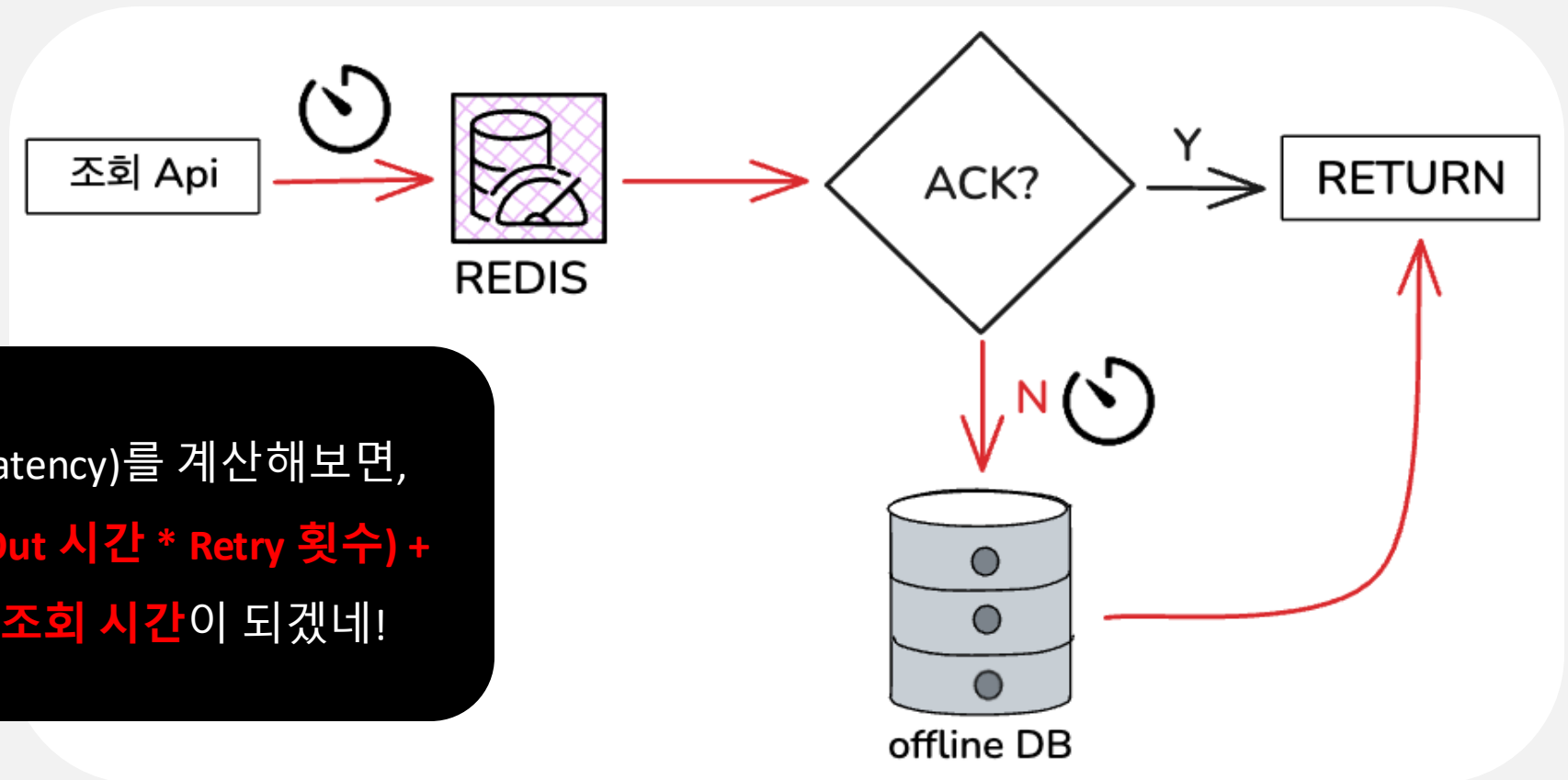
③ 안정성 이슈: 장애 전파 방지 패턴 적용

시스템 장애 감지 시 호출을 차단(OPEN)하고 대체 경로를 사용하게 해주는 CIRCUIT BREAKER



③ 안정성 이슈: 장애 전파 방지 패턴 적용

전통적인 TRY-CATCH 예외처리 방식에서의 REDIS 순단 상황에서는 레이턴시 지연 발생

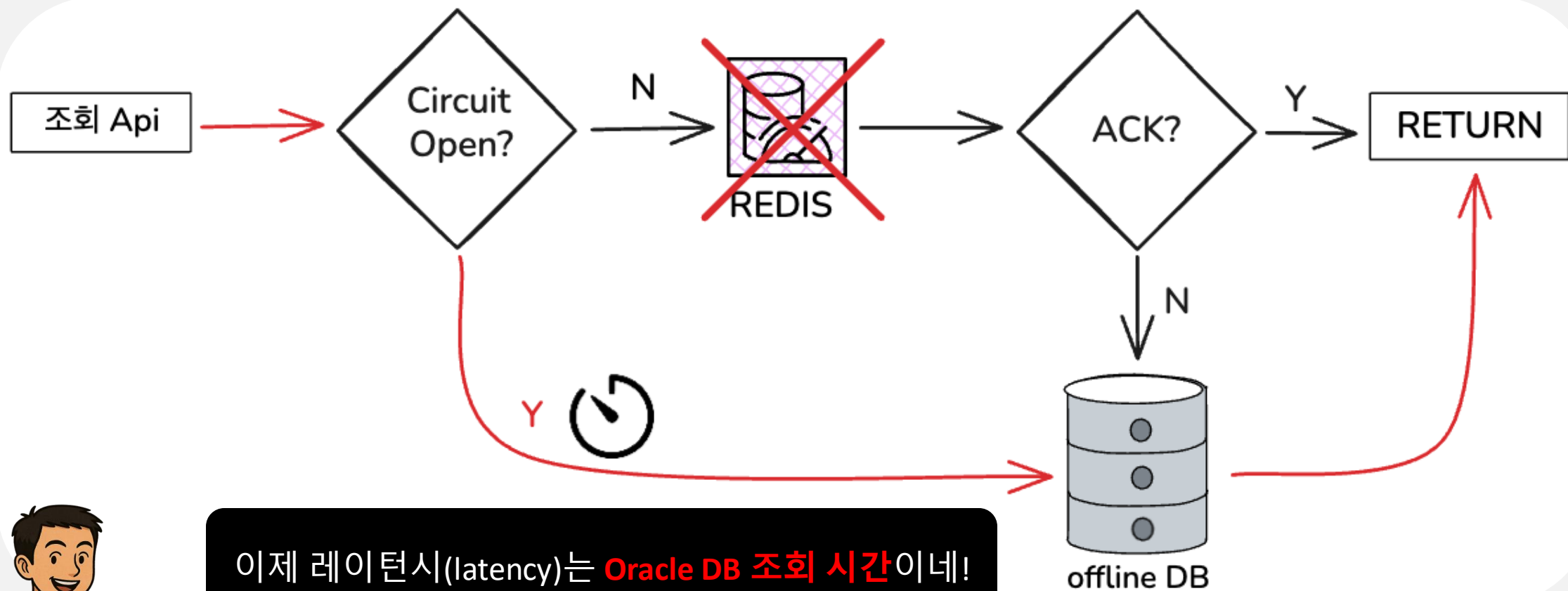


레이턴시(latency)를 계산해보면,
**(Redis TimeOut 시간 * Retry 횟수) +
Oracle DB 조회 시간**이 되겠네!



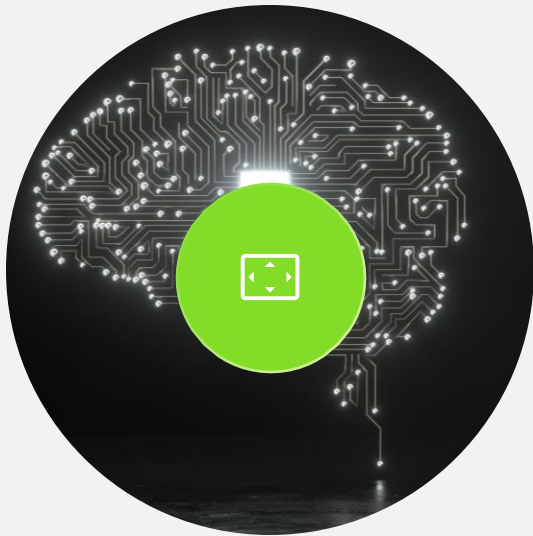
③ 안정성 이슈: 장애 전파 방지 패턴 적용

REDIS 장애 상황에서도 최소한의 응답시간 보장



이제 레이턴시(latency)는 **Oracle DB 조회 시간**이네!

핵심 성과: 연계 서비스 고도화 발판 마련



ORACLE DB 부하 감소

REDIS 사용으로
세일기간 일 평균 3천만번의
Oracle 조회 액션 제거



조회 성능

조회 상황별 데이터셋 설계로
조회 API 성능 10배 향상



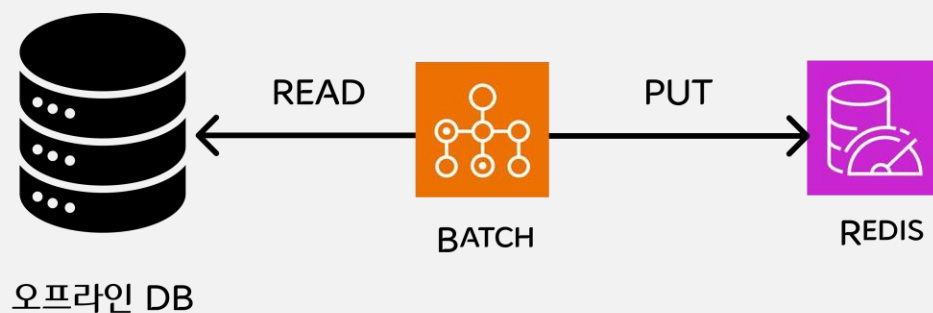
안정성

Circuit Breaker를 사용해
안정적인 API 제공

재고 싱크 지연 해소를 위해 남은 숙제

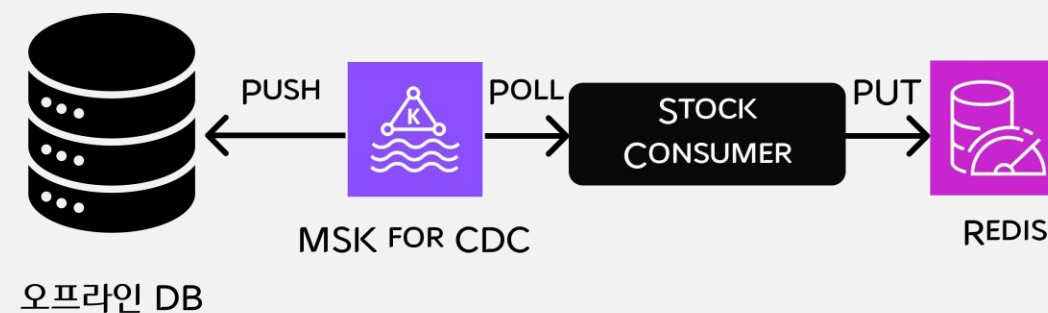
배치기반 재고 싱크는 EVENT-DRIVEN/CDC로 대체

현재



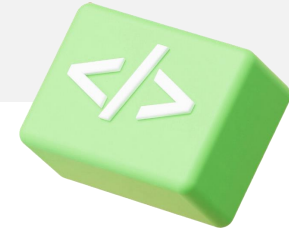
Batch 기반 Oracle DB, Redis 싱크로 인한
재고 싱크 지연

TOBE



이벤트 드라이븐/CDC로 대체하기

기술 선택보다 중요한 것: 도메인 이해와 구조 설계



도메인과 고객 경험을 이해한 설계 없이는,
어떤 기술도 성능을 보장하지 않습니다





WHAT'S NEXT?

글로벌 뷰티&헬스 트렌드 리딩 플랫폼으로 뻗어나가고 있는 올리브영

앞으로 더 많은 국가, 더 다양한 고객을 만나기 위해 올리브영의 물류 시스템도 발맞춰 계속 진화해나갈 예정입니다.



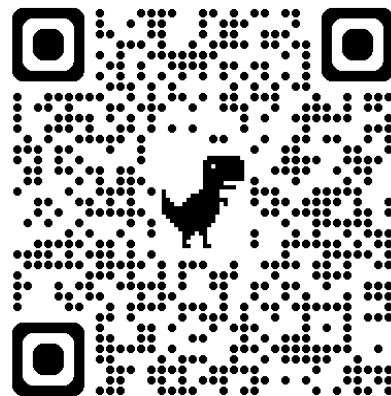
OLIVE YOUNG

OY.TECH@OLIVEYOUNG.CO.KR

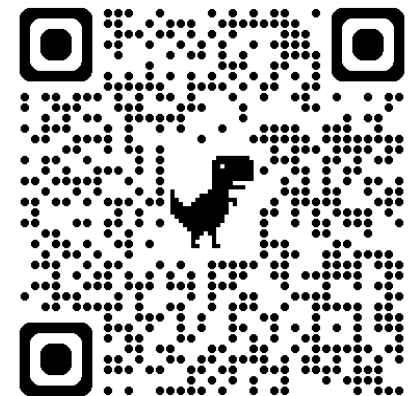
OY TECH BLOG



OY CAREERS



OY LINKEDIN



OLIVE YOUNG

THANK YOU

감사합니다:)

